



Audio Computing Lab

Classification and Music Information Retrieval

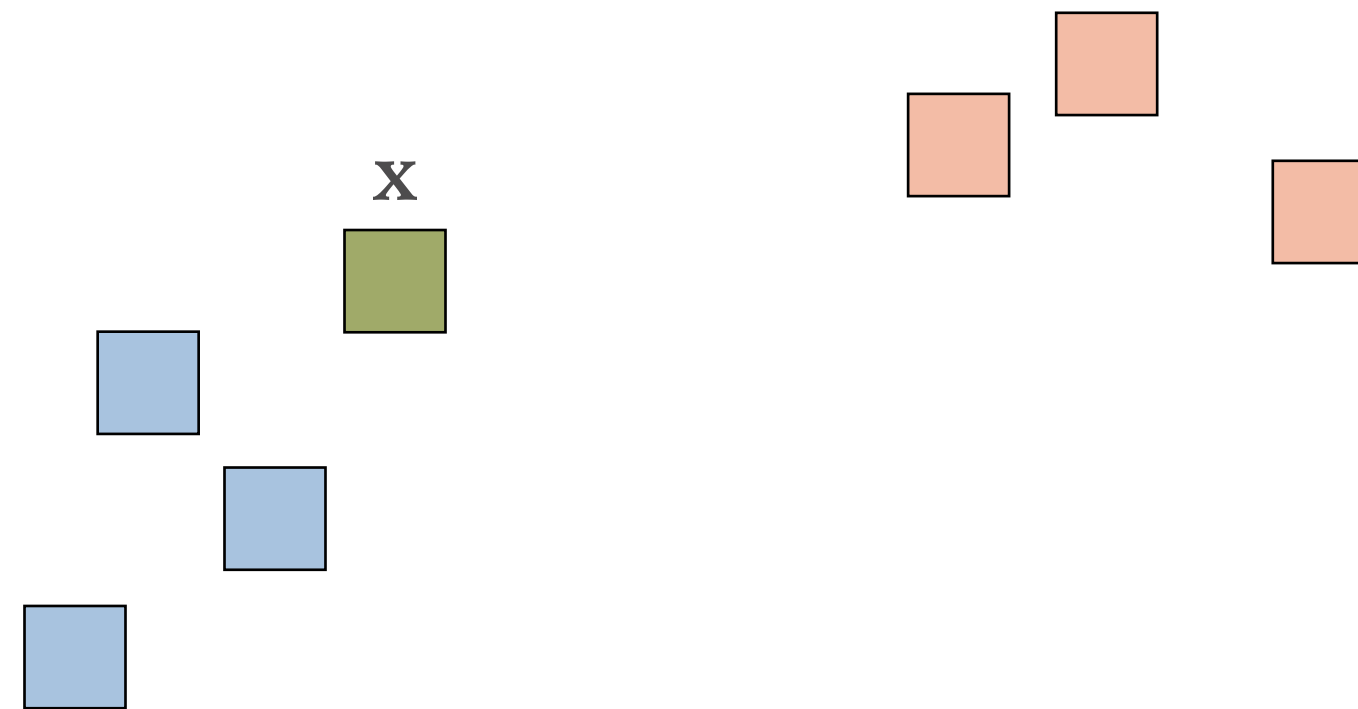
Paris Smaragdis
paris@illinois.edu
paris.cs.illinois.edu

Today's lecture

- Classification basics
 - Basic Gaussian and Gaussian Mixture classifiers
 - Audio features
- Music Information Retrieval (MIR) tasks
 - Genre classification
 - Audio fingerprinting

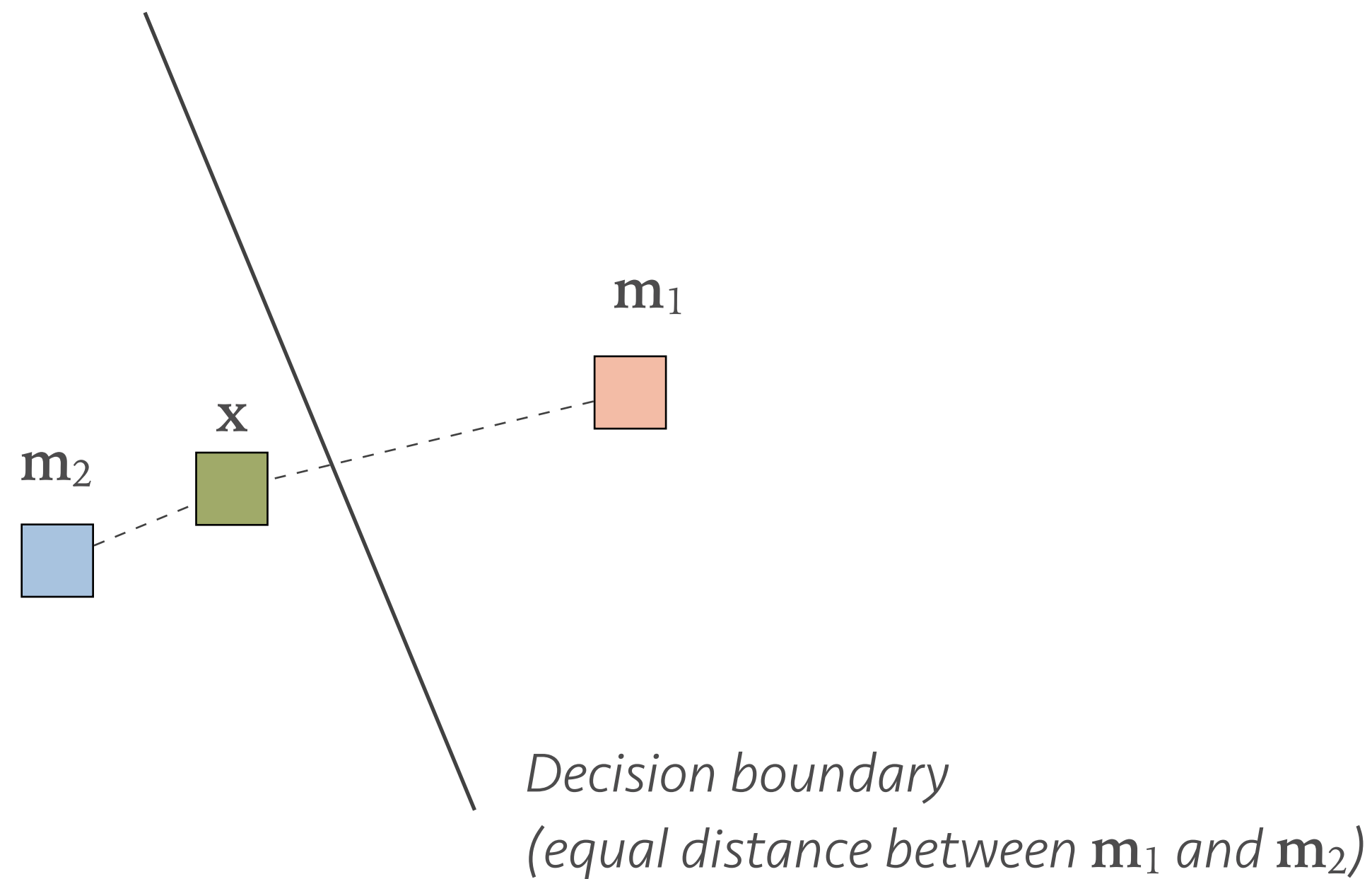
Classification

- How do we automate decisions?
 - Here's the usual setup



Nearest neighbors

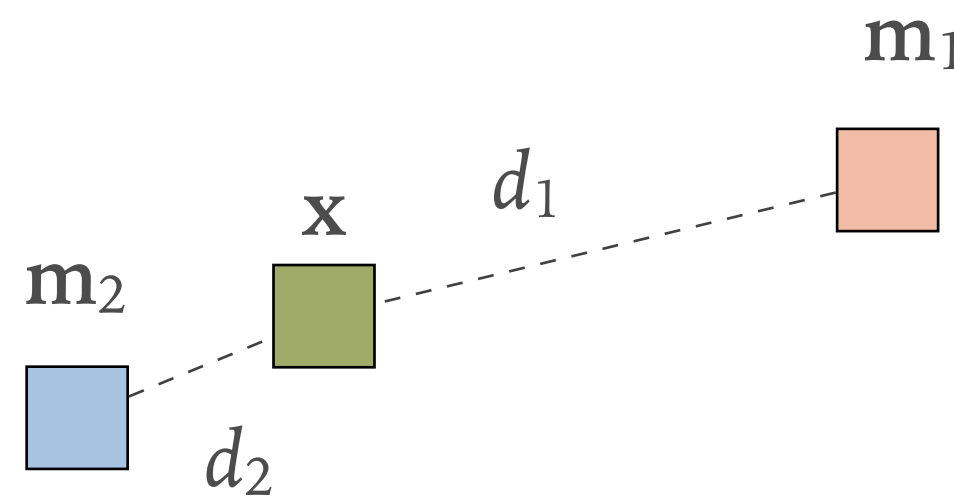
- We can classify based on the nearest neighbors



- Can we quantify similarity though?

Nearest neighbors

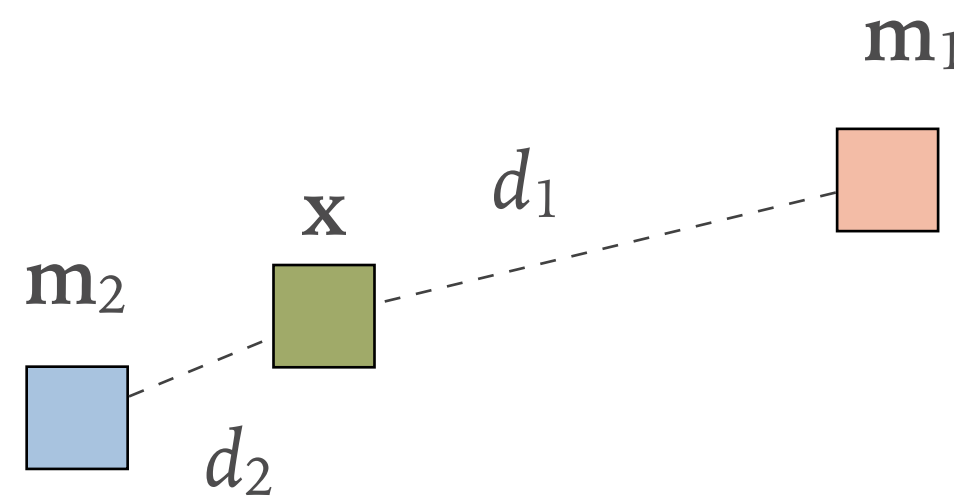
- We can use distance to quantify similarity



$$d_i = \sqrt{(x_1 - m_{i,1})^2 + (x_2 - m_{i,2})^2} = ||\mathbf{x} - \mathbf{m}||$$

Making a “soft” decision

- What if I didn't want to classify?
 - What if I wanted a “degree of belief”
 - Or a class probability



- How do we get that?

From a distance to a probability

- If the distance is 0 $\rightarrow$ the probability is 1
- If the distance is ∞ $\rightarrow$ the probability is 0
- How do we make a function for this mapping?
 - Over all possible distances?

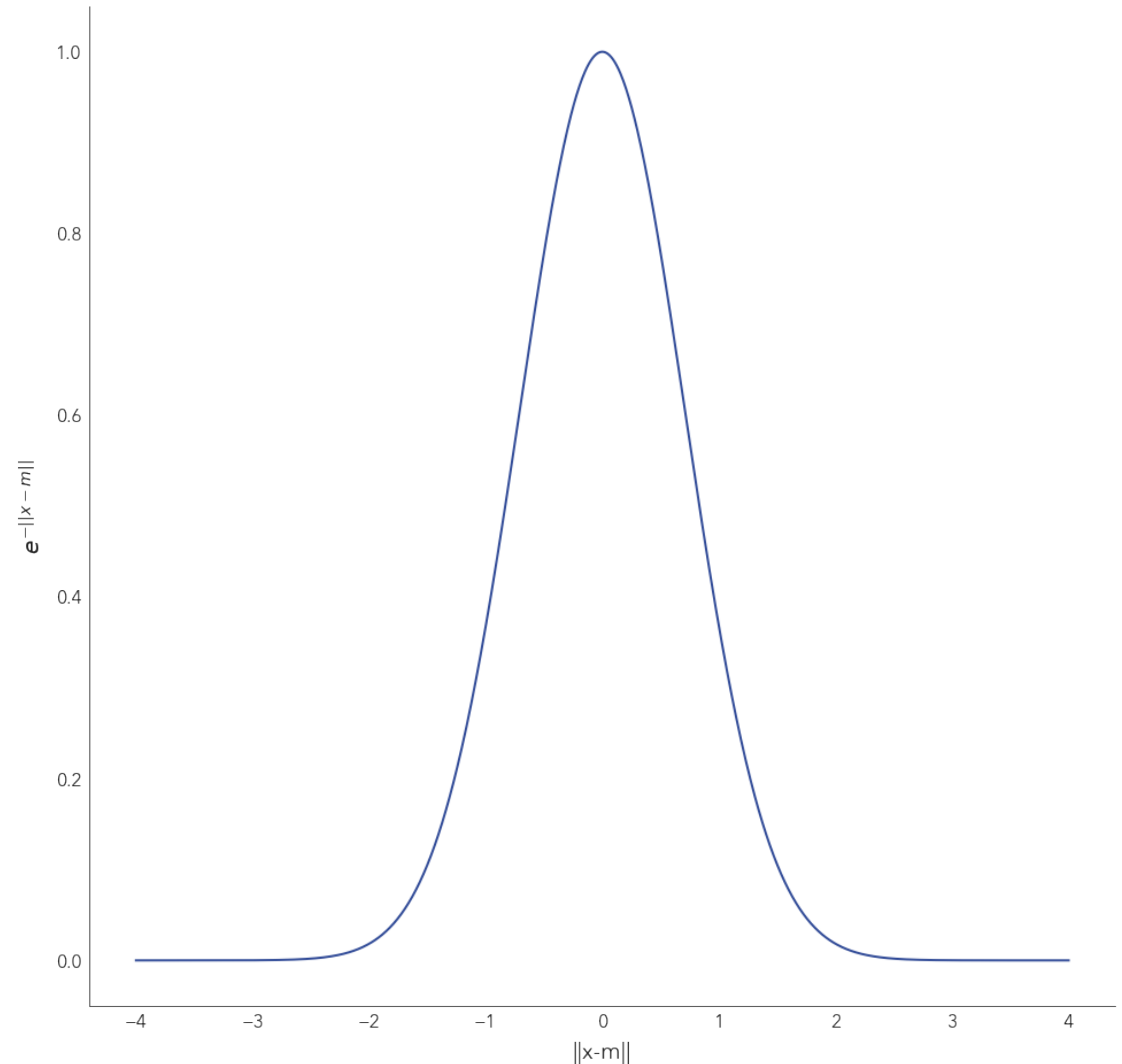
First crack at it

- Use exponentiation

$$p = e^{-\|\mathbf{x}-\mathbf{m}\|}$$

- For large distance
 - We get a small p
- For small distance
 - We get a large p

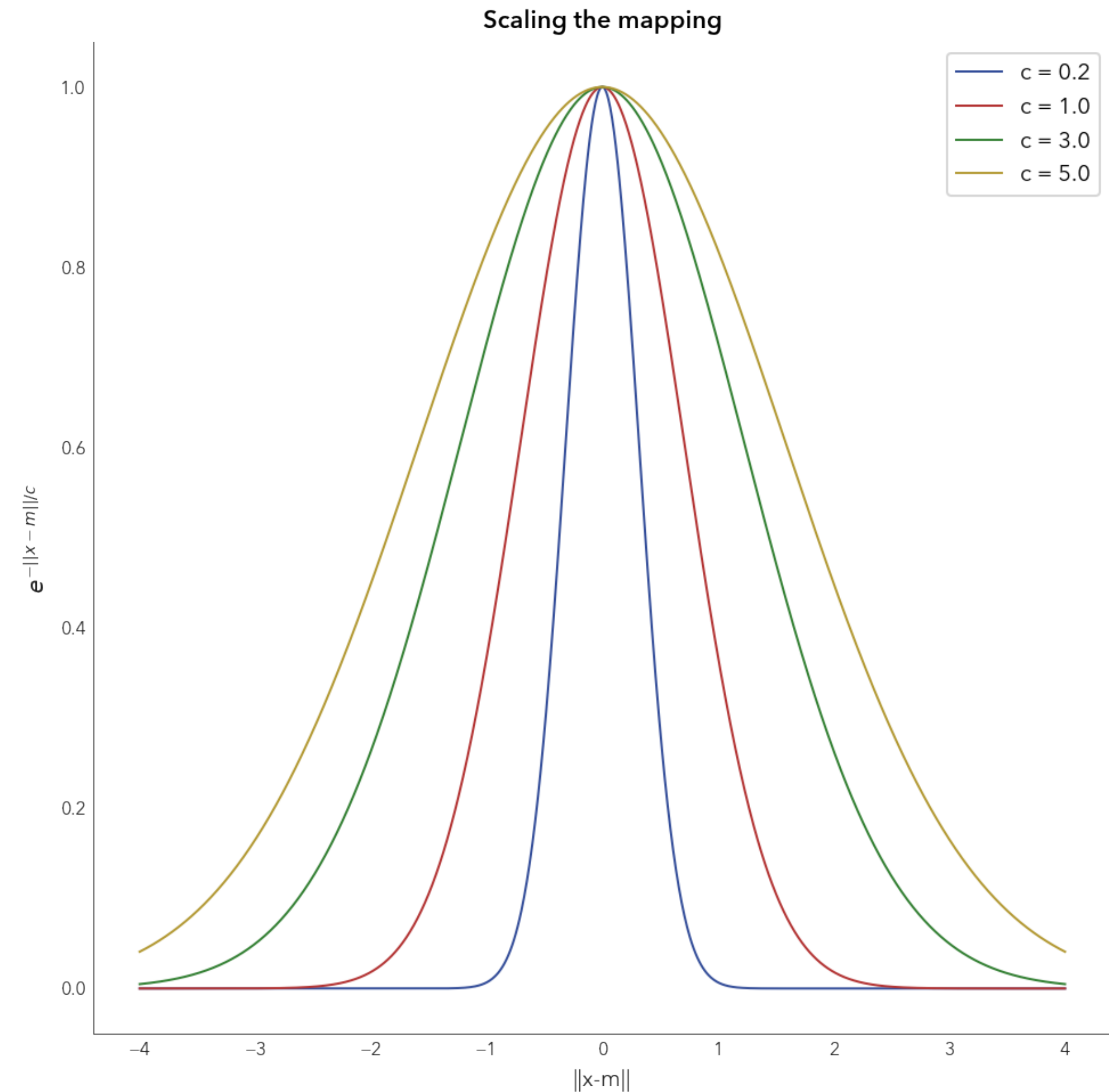
Mapping distance to a probability-like measure



Adding a scaling factor

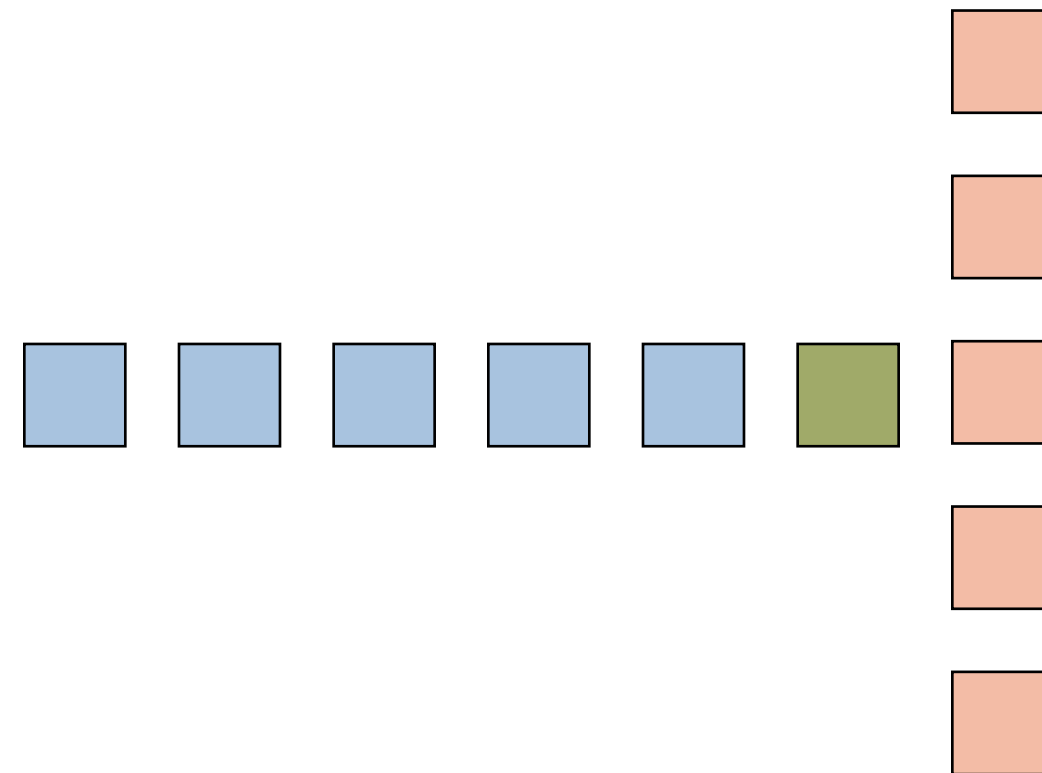
- Not all cases are the same
 - E.g. 5 lb of weight distance
 - Insignificant for a car's weight
 - Very significant for buying sugar
- Adding a factor for scale

$$p = e^{-\frac{\|x-m\|}{c}}$$



One more problem

- Not all dimensions are equal



Treating each dimension differently

- Use a different scaling for each dimension, e.g.:

$$p = e^{-\left(\frac{(x_1 - m_1)^2}{c_1} + \frac{(x_2 - m_2)^2}{c_2} \right)}$$

- Or use a more general form:

$$p = e^{-(\mathbf{x} - \mathbf{m})^\top \cdot \mathbf{C}^{-1} \cdot (\mathbf{x} - \mathbf{m})} = e^{-c_1 d_1^2 - c_2 d_2^2 - 2c_0 d_1 d_2}$$

$$d_i = x_i - m_i, \quad \mathbf{C}^{-1} = \begin{bmatrix} c_1 & c_0 \\ c_0 & c_2 \end{bmatrix}$$

Note: I use the convention that $\mathbf{x}$ is a row vector

* Most often we will use a diagonal $\mathbf{C}$

The Gaussian distribution

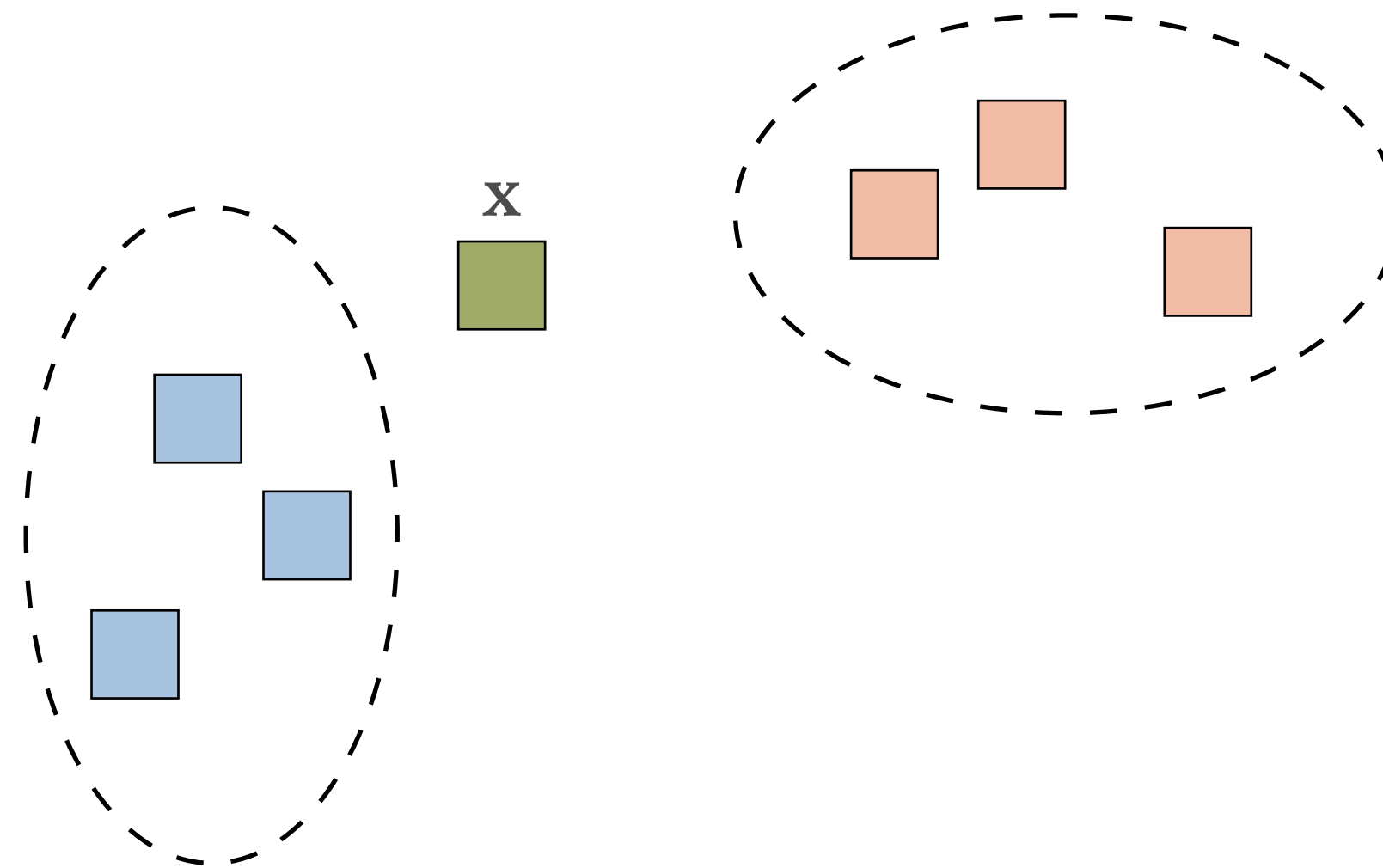
- All this leads to the Gaussian distribution
 - We also need to normalize so it integrates to 1:

$$P(\mathbf{x}|\mathbf{m}, \mathbf{C}) = \frac{1}{\underbrace{\sqrt{(2\pi)^k |\mathbf{C}|}}_{\text{Normalization factor}}} e^{-\frac{1}{2} (\mathbf{x}-\mathbf{m})^\top \cdot \mathbf{C}^{-1} \cdot (\mathbf{x}-\mathbf{m})}$$

- $\mathbf{m}$ is the mean
- $\mathbf{C}$ is the covariance
- k is the number of dimensions

Gaussian class models

- We can now describe data using Gaussian models



- But how do we get the models given training data?

Learning Gaussian parameters

- We need to get a mean $\mathbf{m}$ and a covariance $\mathbf{C}$
 - For the mean:

$$\mathbf{m} = \frac{1}{N} \sum \mathbf{x}_i$$

- For the covariance:

$$\mathbf{C} = \frac{1}{N-1} \sum_i (\mathbf{x}_i - \mathbf{m})^\top \cdot (\mathbf{x}_i - \mathbf{m})$$

Now we can make classifiers

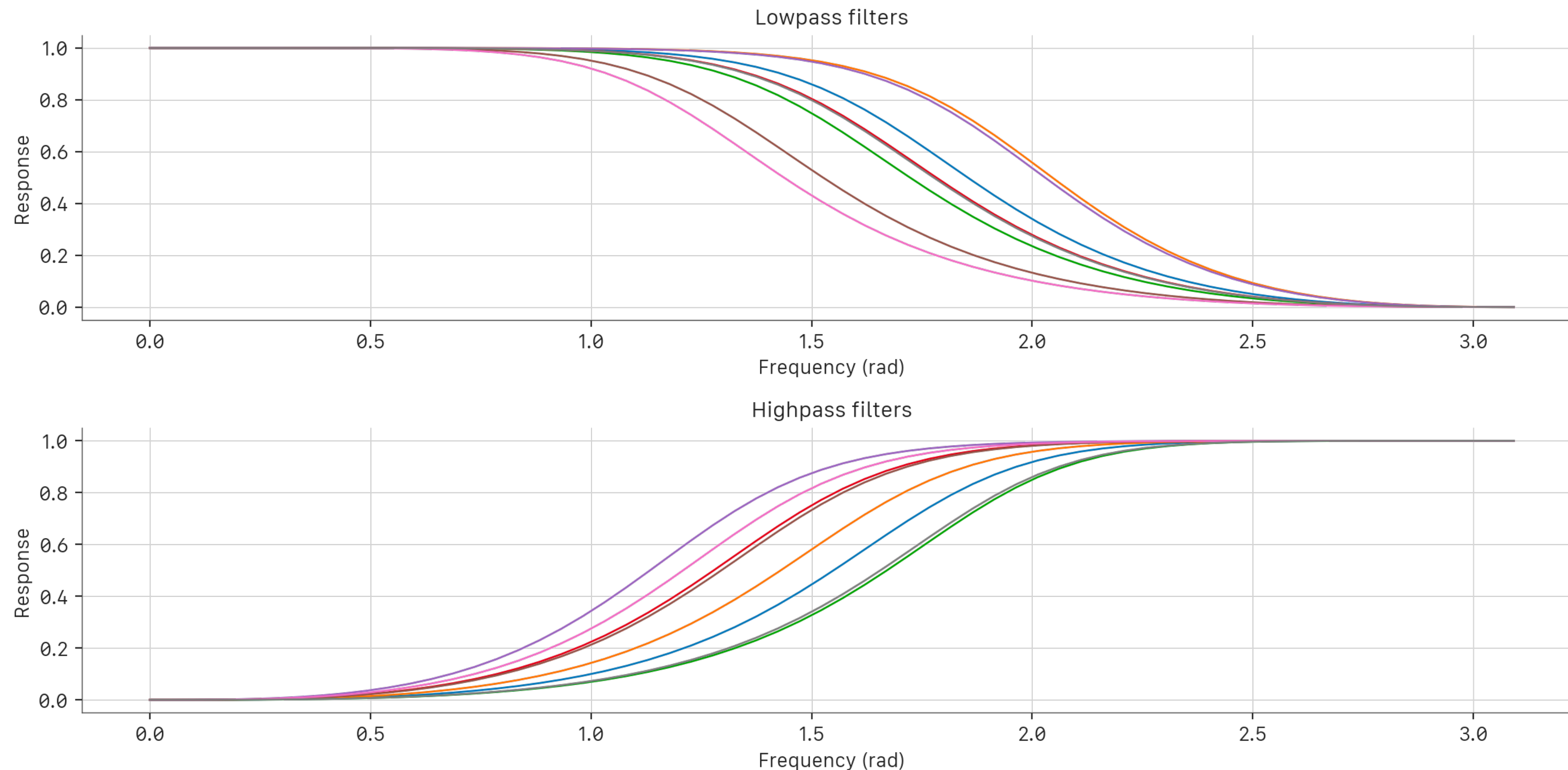
- Instead of nearest neighbors we will use probabilities
 - Which are always much better when making classifiers!
- We compute a belief of class assignment
 - For each data sample, each model gives us a likelihood
 - Class that provides largest likelihood claims the sample

The classification process

- We obtain example of classes
 - E.g. collect sample of classes of sounds
- We make a model for each class
 - Learn the Gaussian model parameters of, e.g., their spectra
- We assign all new input data to a class
 - Based on which class is most likely

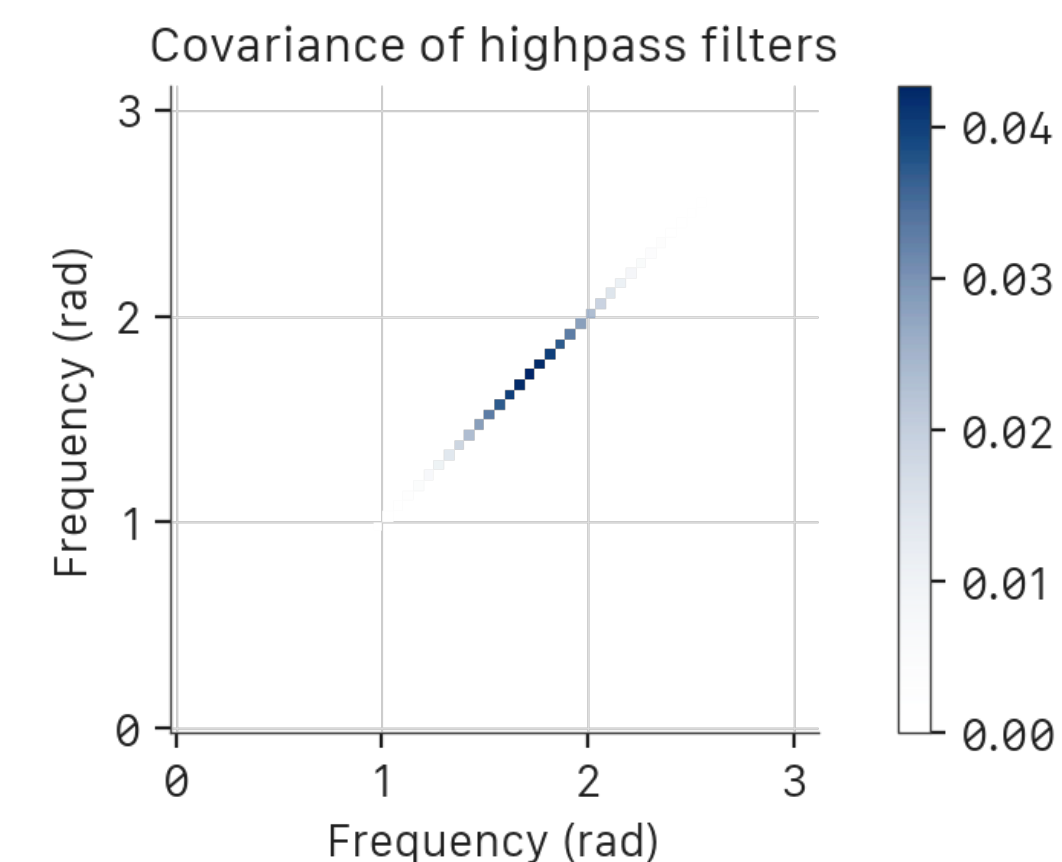
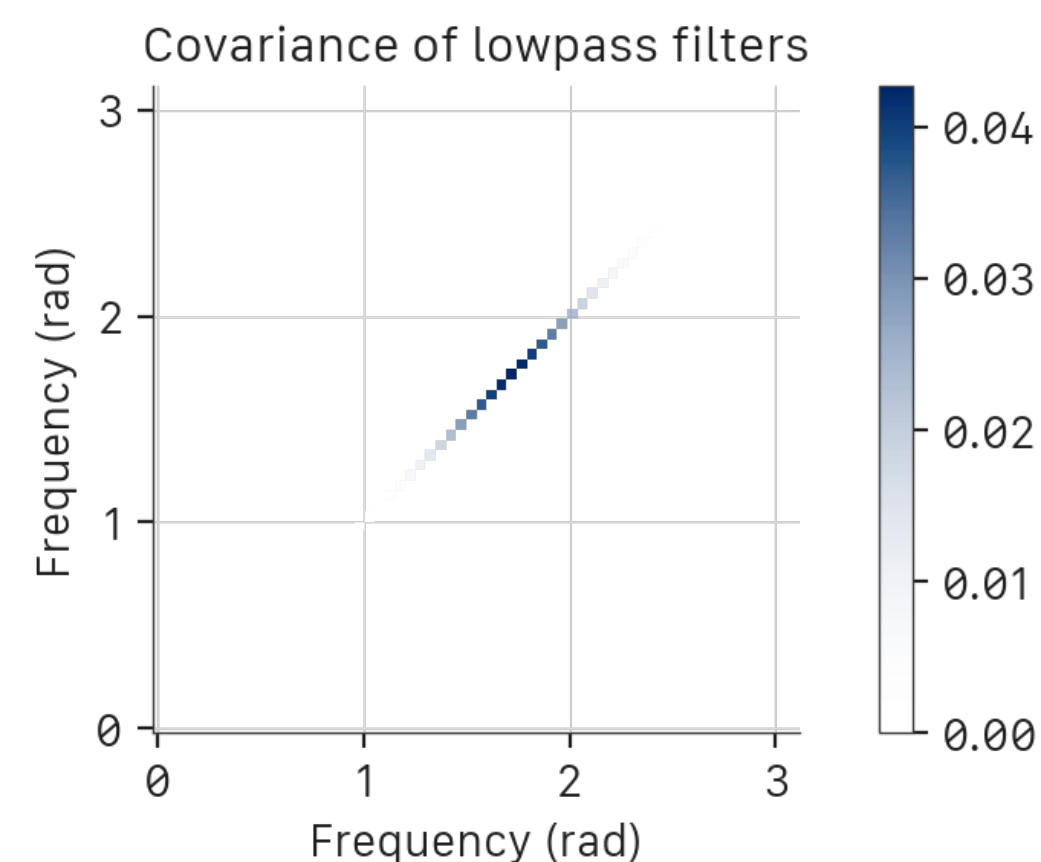
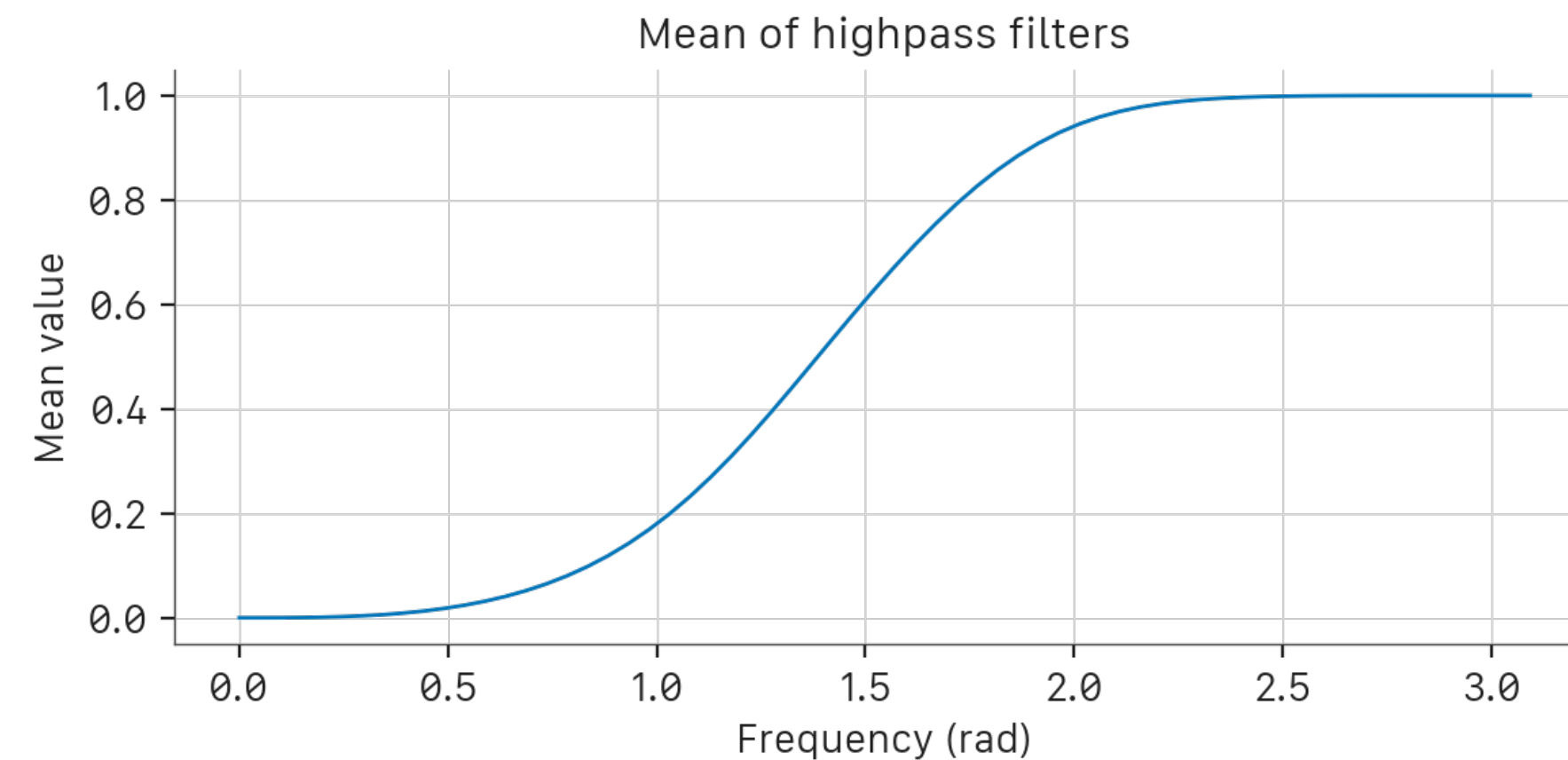
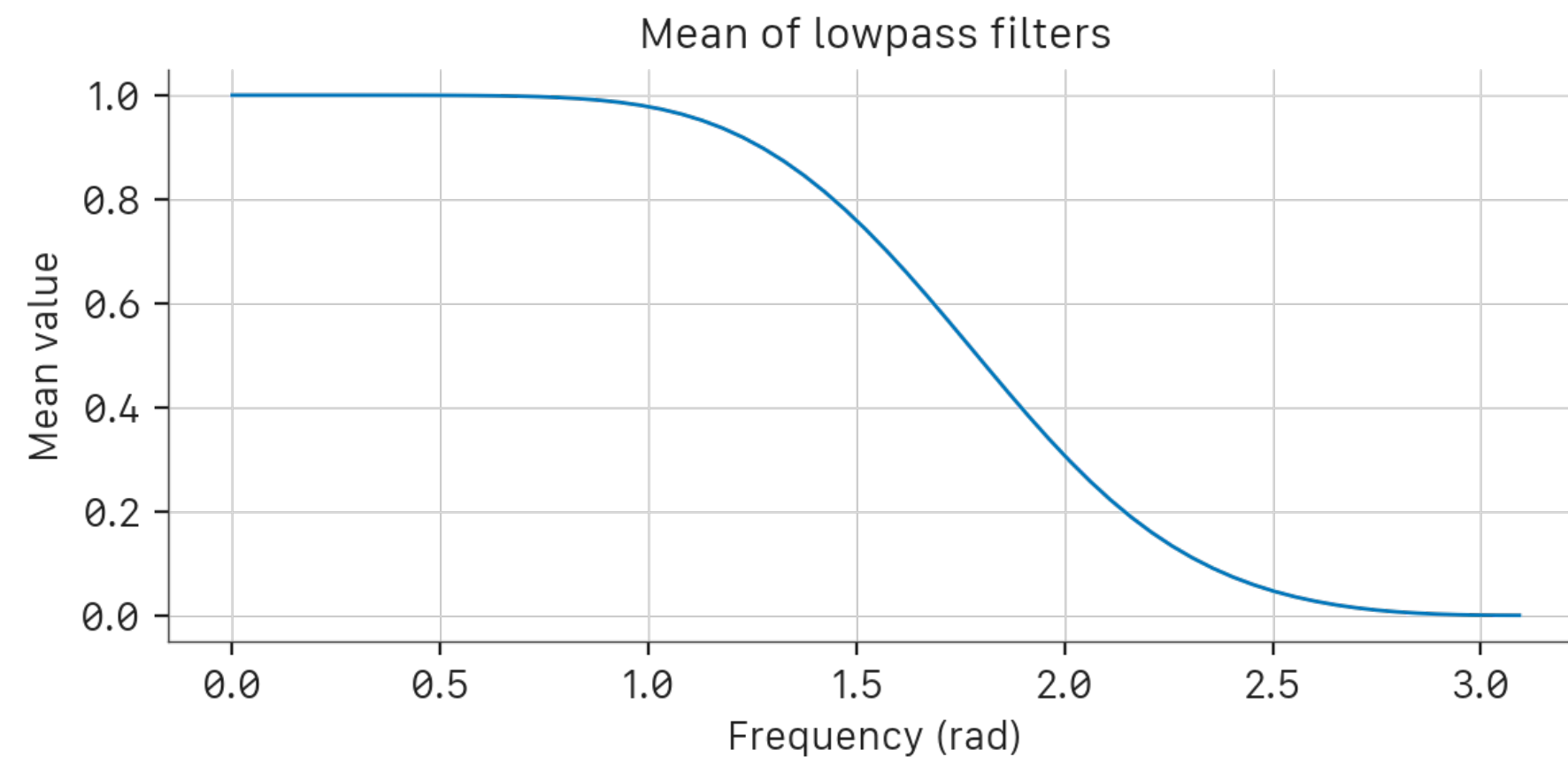
Simple example

- Classifying filter responses
 - Automatically classify filter as lowpass or highpass



Step 1: Learn the models

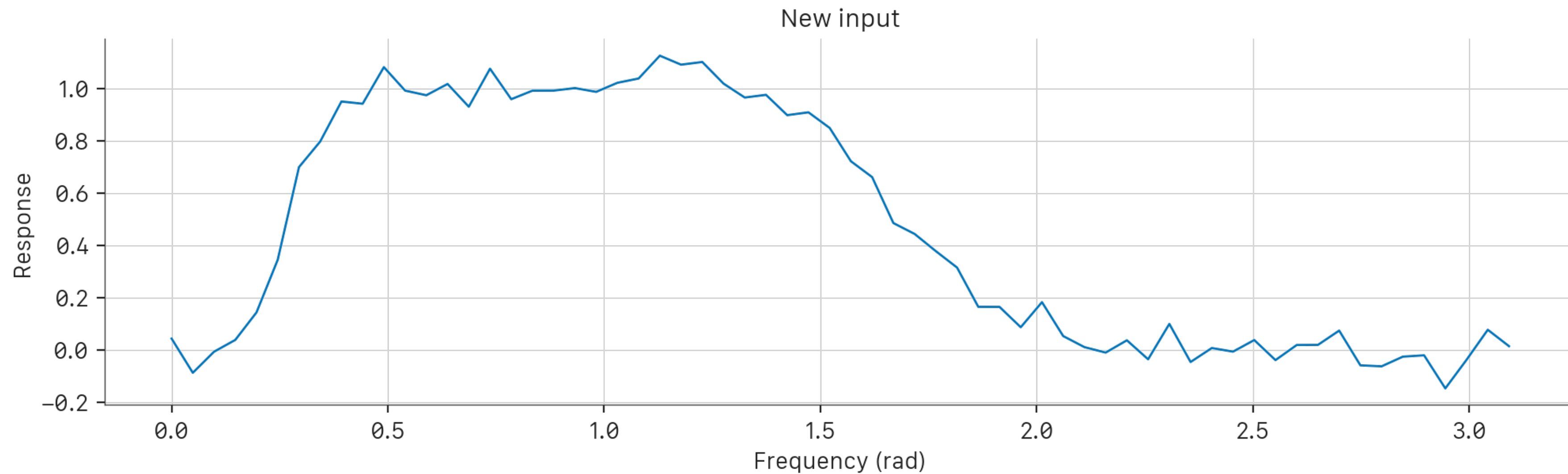
- Estimate mean and covariance of each class
 - Looking at them they should make sense



** Only taking the diagonal of the covariance here*

Step 2: Get likelihoods for new data

- Which class does this new input belong to?



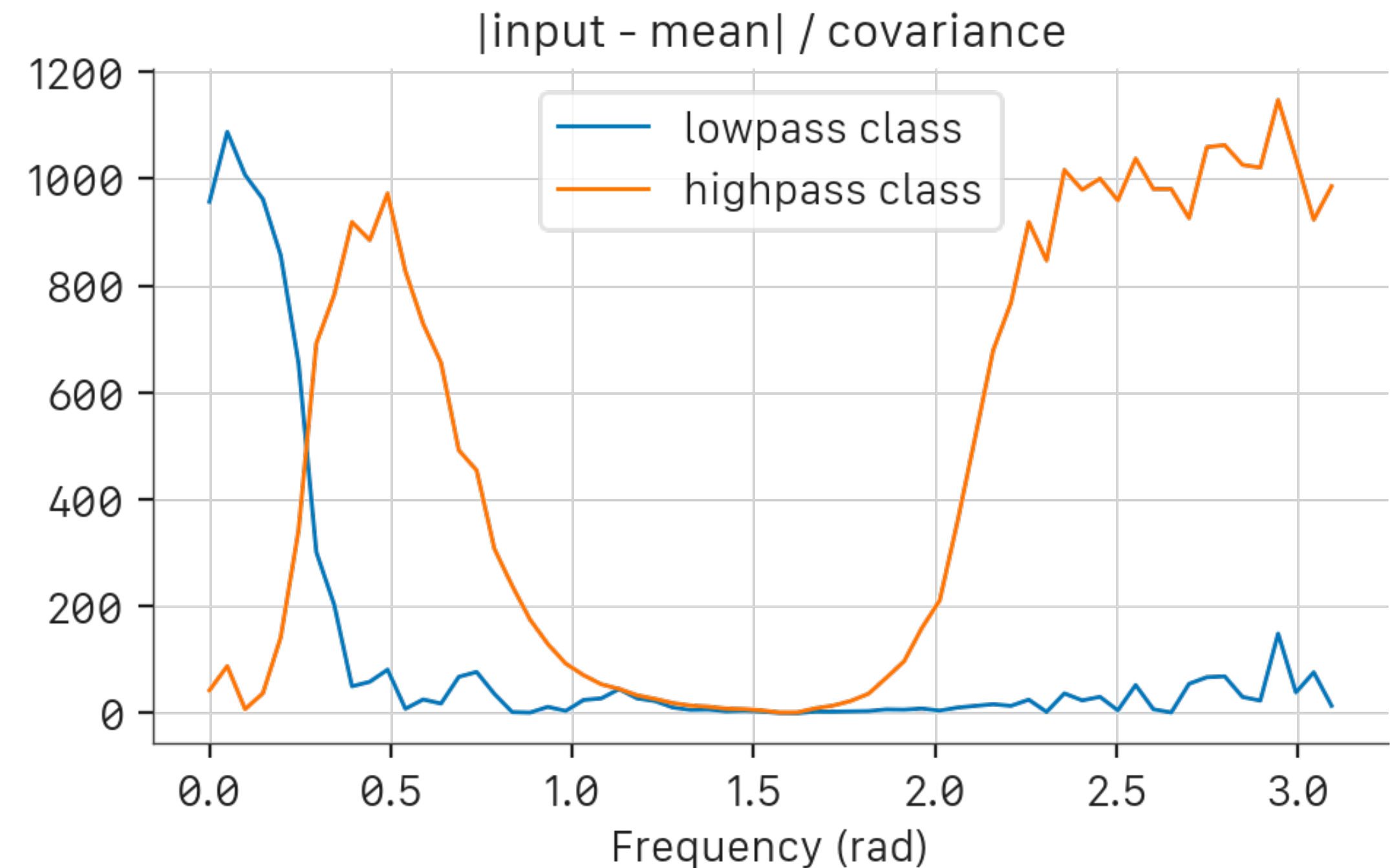
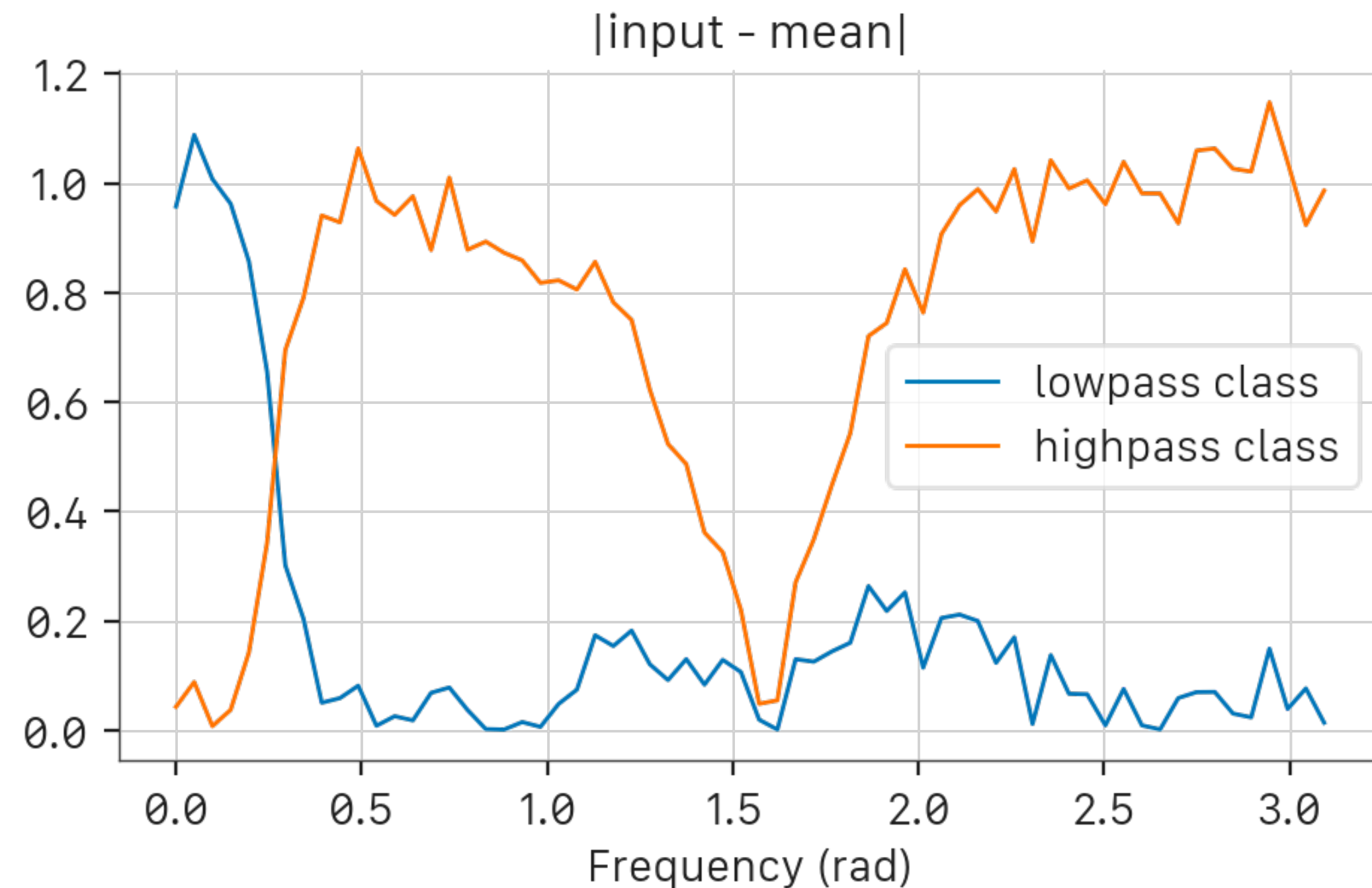
- Get class likelihoods:

$$P(x|\mathbf{m}_1, \mathbf{C}_1) > P(x|\mathbf{m}_2, \mathbf{C}_2)$$

- Bigger likelihood wins

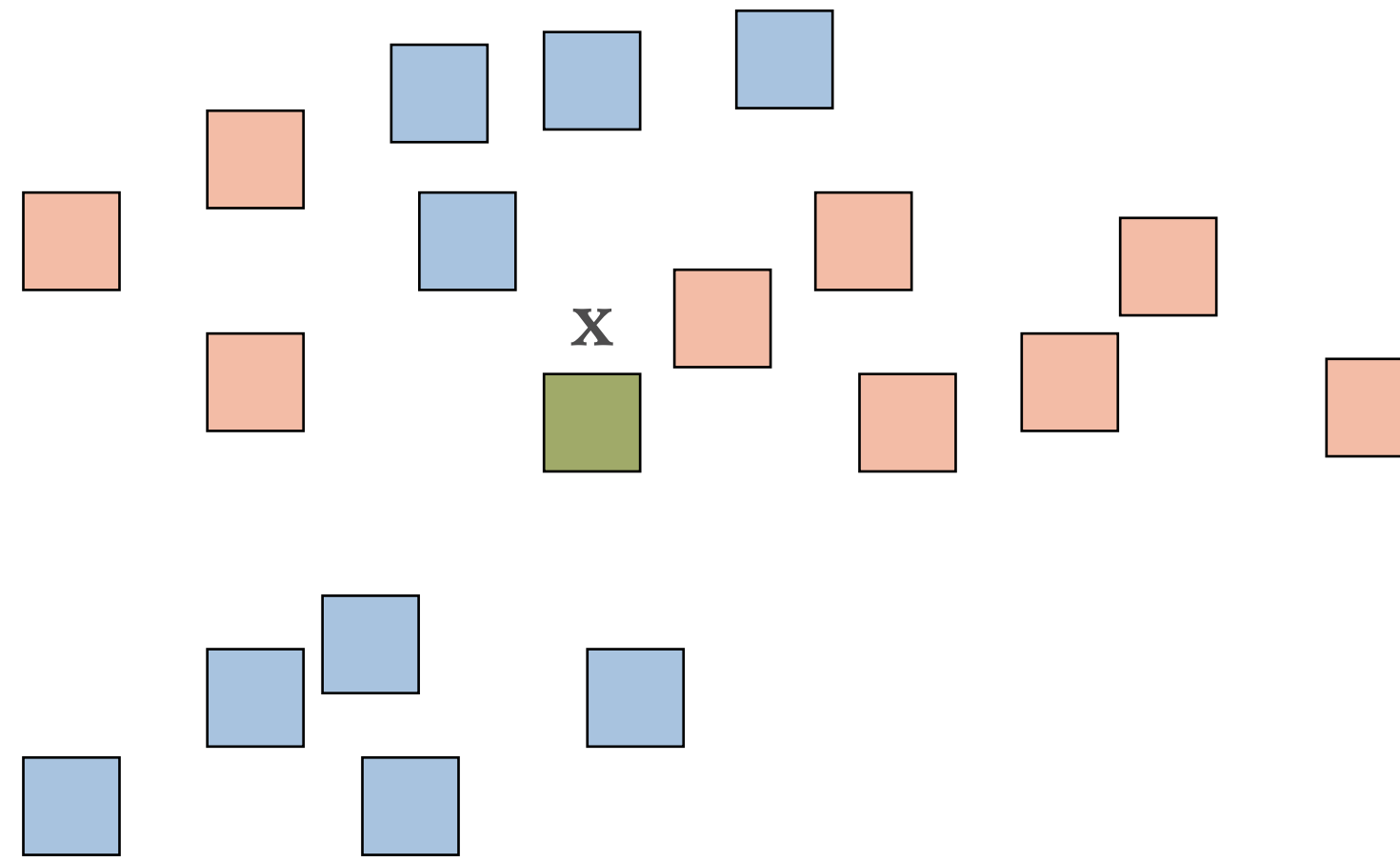
How does that look like?

- Removing the means shows overall class error
- Using covariance scales down ambiguous regions



What about more complex data?

- Life isn't always very Gaussian

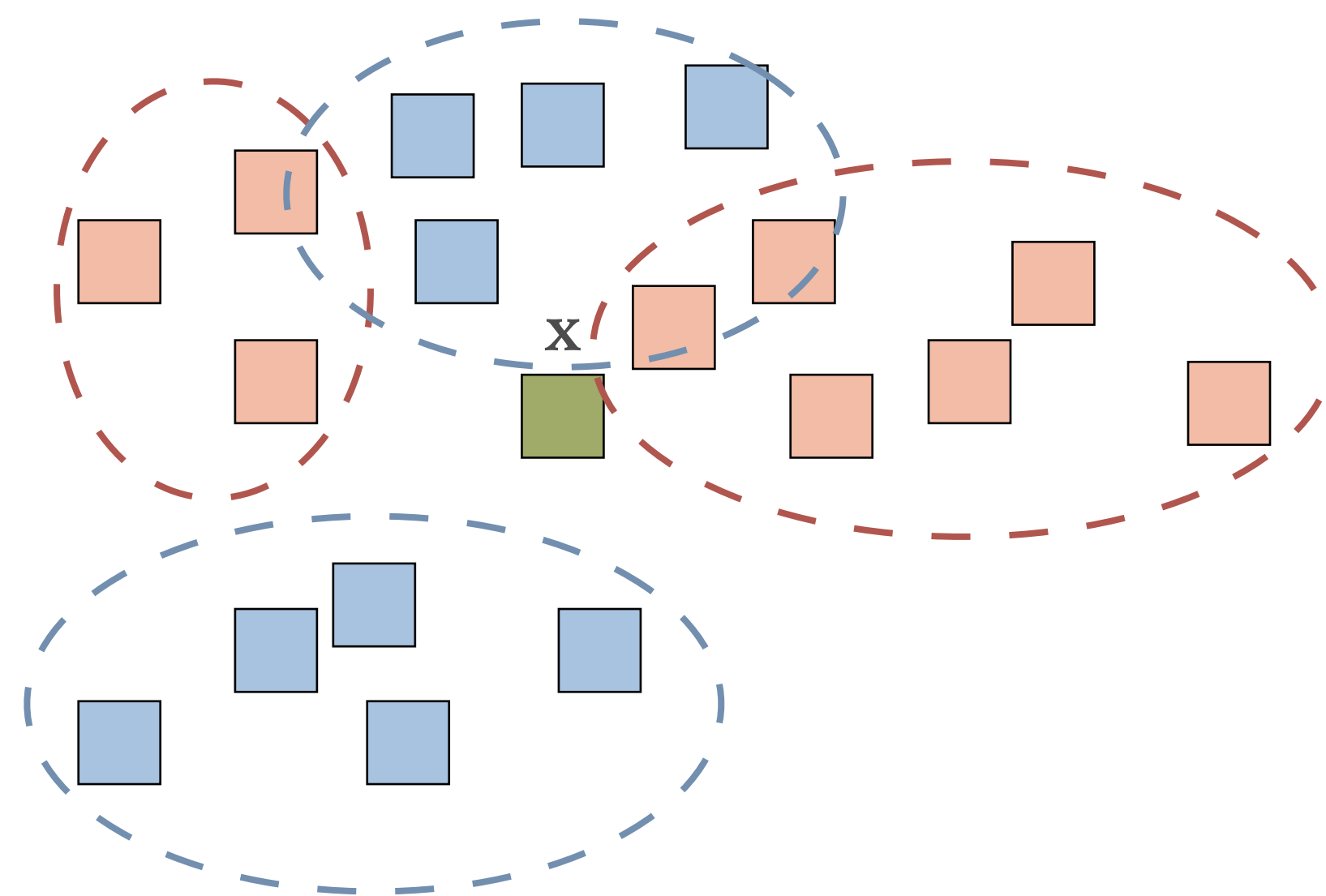


- We need to describe our classes using a more powerful model

Gaussian Mixture Models

- Model each class using multiple Gaussians

$$P(\mathbf{x}|\text{class}_j) = \sum_i^N p_{i,j} \mathcal{N}(\mathbf{x}|\mathbf{m}_{i,j}, \mathbf{C}_{i,j})$$



- To classify with these
 - Same process as before, but with a GMM instead of a Gaussian distribution for each class
 - Learning the models is more complex, but there's lots of software for it

Classifying sound recordings

- Perform an STFT to get a set of spectra $\mathbf{f}_t$
- Get each $\mathbf{f}_t$'s likelihood $L_{c,t}$ for class c

$$L_{c,t} = \text{model}_c(\mathbf{f}_t)$$

- This is the class likelihood for each analysis frame
- Combine likelihoods over time span of interest

$$L_{c,t_1 \leftrightarrow t_2} = \sum_{i=t_1}^{t_2} L_{c,i}$$

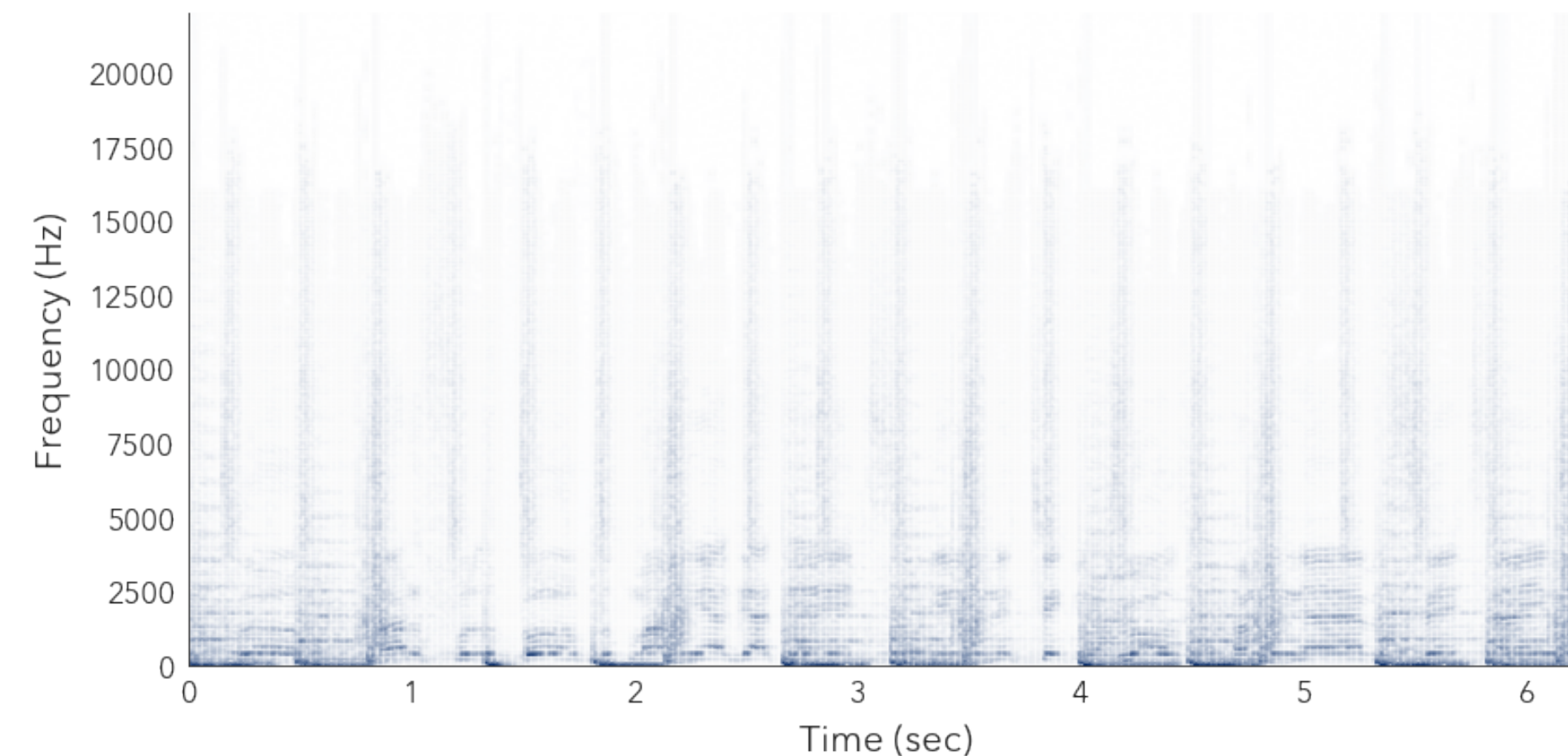
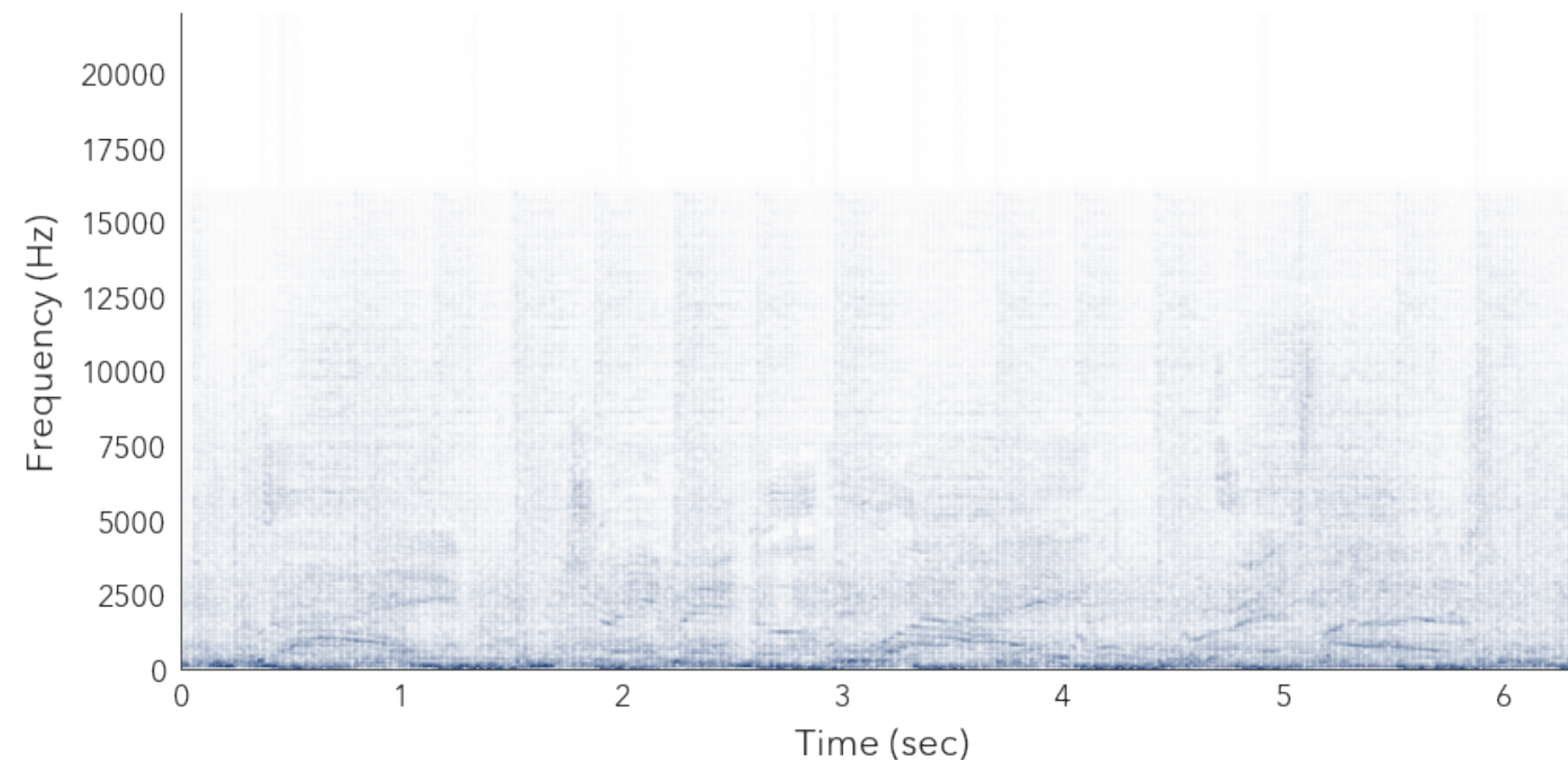
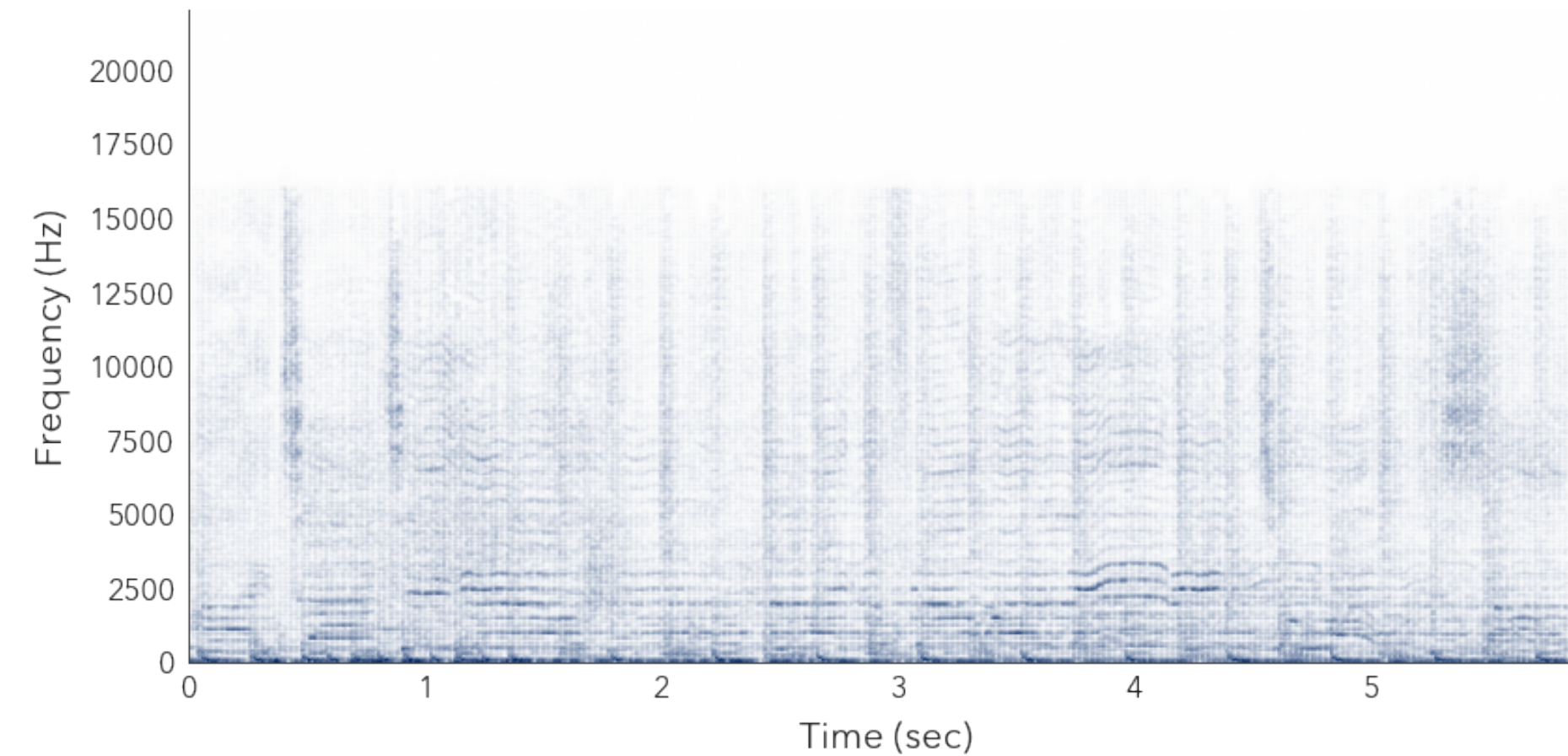
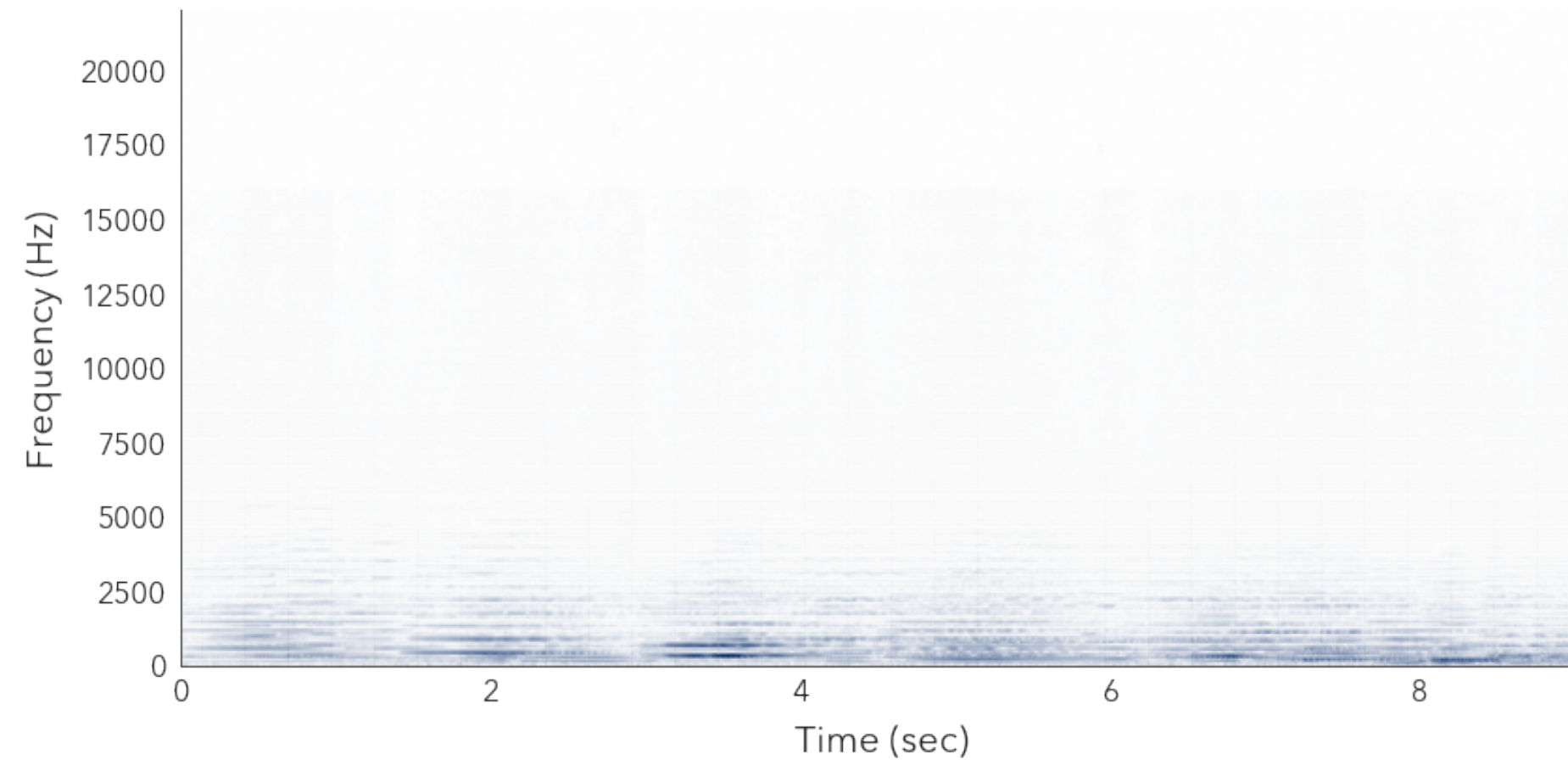
If you compute probabilities you multiply them to combine them. But it is numerically better to just use the log probabilities and sum them

Musical genre identification

- Identify the style of a music recording
 - Jazz, rock, rap, classical, etc.
- Let's use some Gaussians!

Basic premise

- Different genres sound different
 - i.e. their audio features are different classes

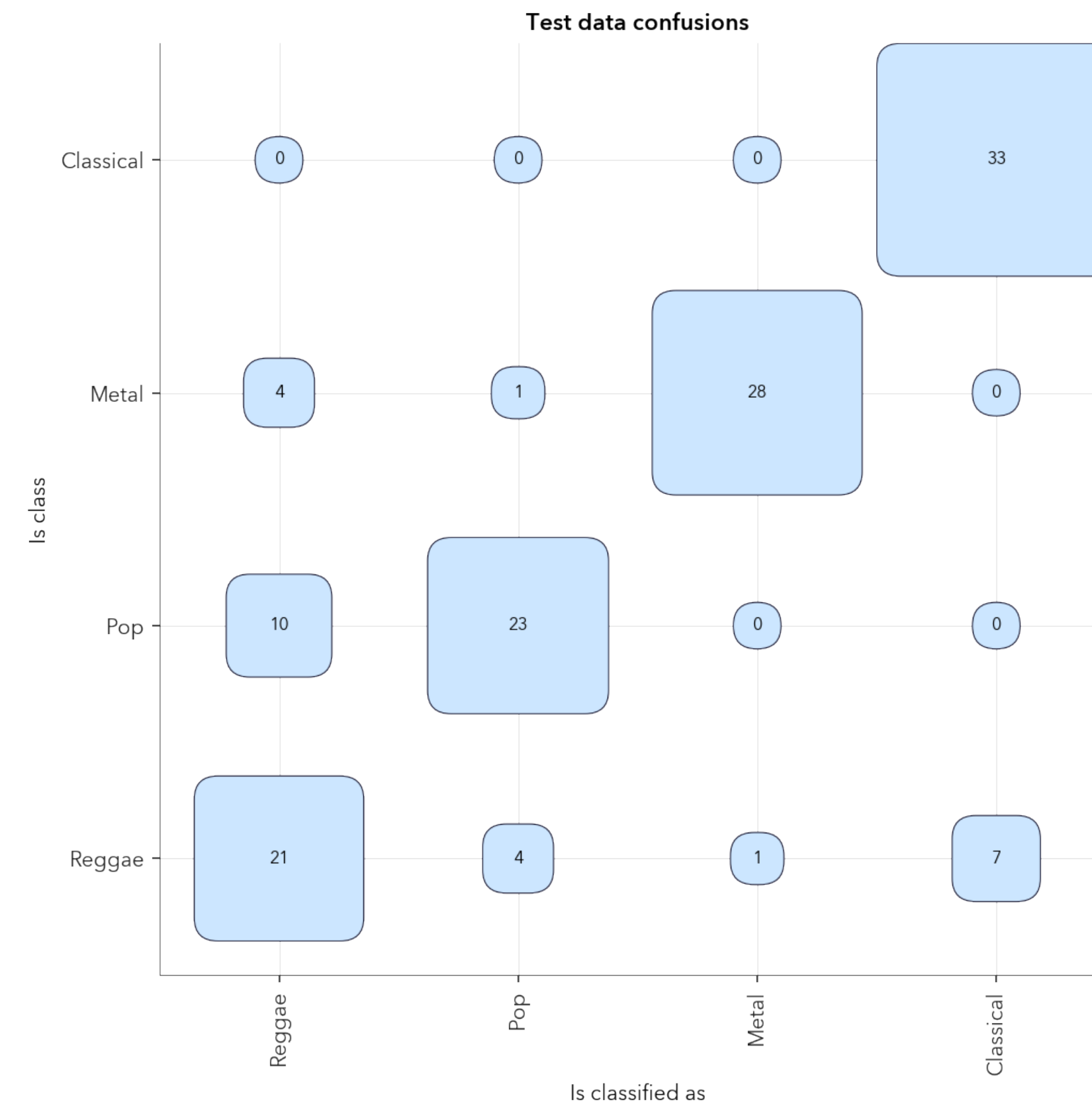
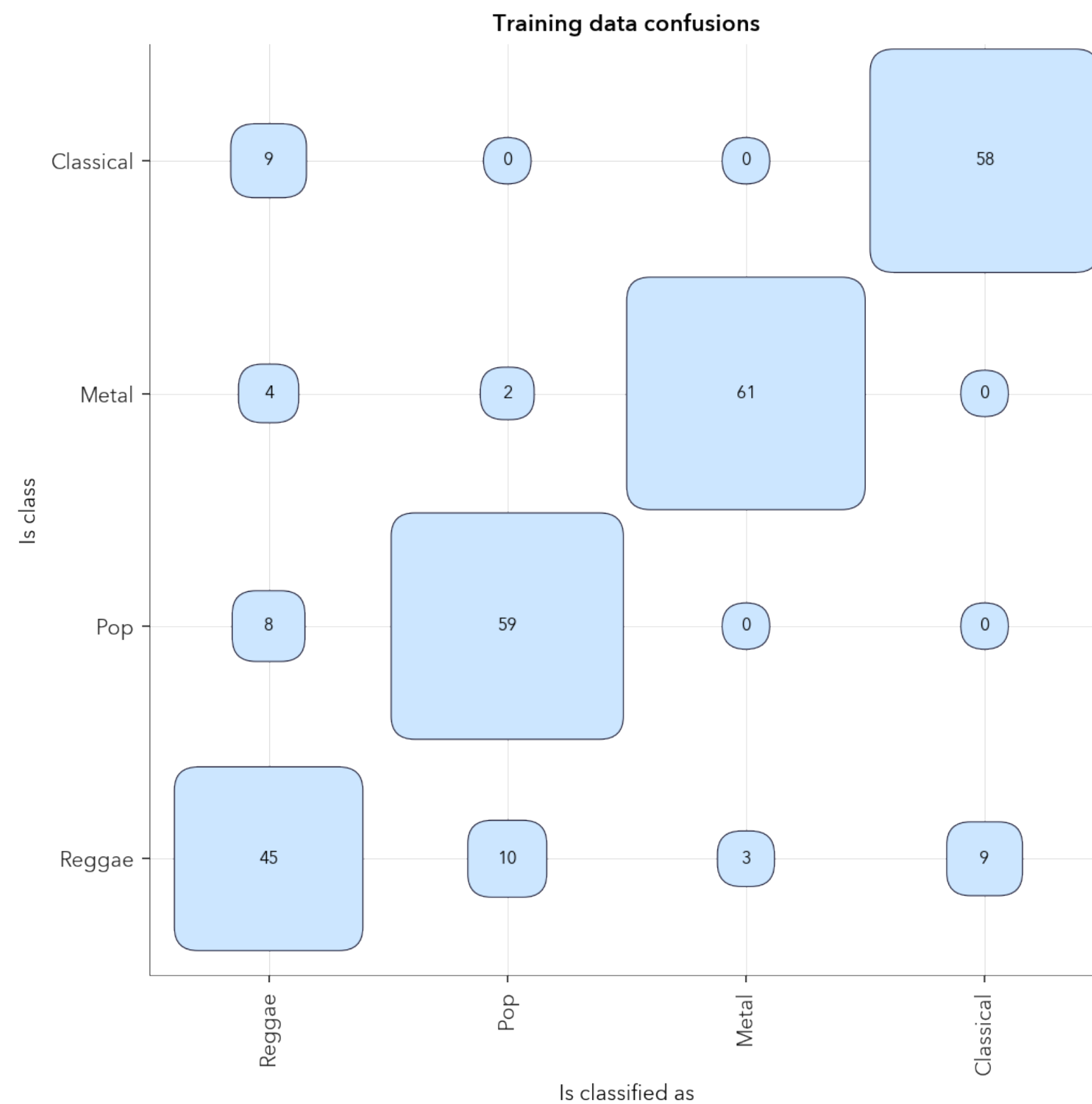


Making a simple classifier

- Step 1: Collect training data for all classes
 - i.e. get magnitude spectrograms for each sound class
- Step 2: Estimate all class means and covariances
 - This will give you the class models
- Step 3: Evaluate on unseen data to check classifier
 - Should perform better than chance guessing

Simple example

- Learn a Reggae/Pop/Metal/Classical classifier
 - Use a 3-Gaussian GMM on magnitude spectra



More elaborate features for music

- Spectrograms are a little too detailed
- We sometimes need to derive coarser features
 - Overall sound and rhythm quantities

Timbral features

- Timbre: “Quality of sound”
 - What is not pitch or amplitude
- A function of the spectral shape
 - “bright”, “hissy”, “muted”, “nasal”, “boomy”, ...

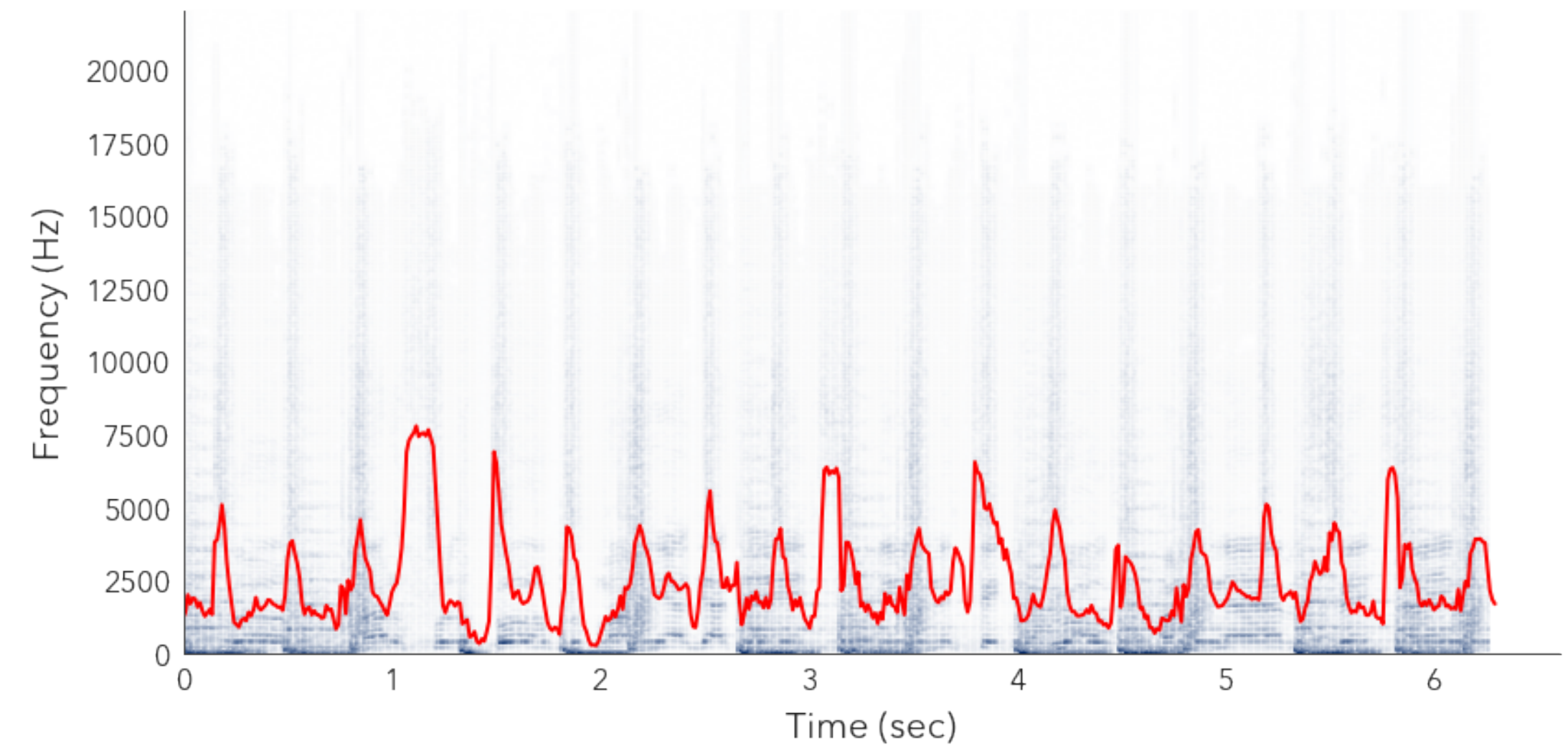
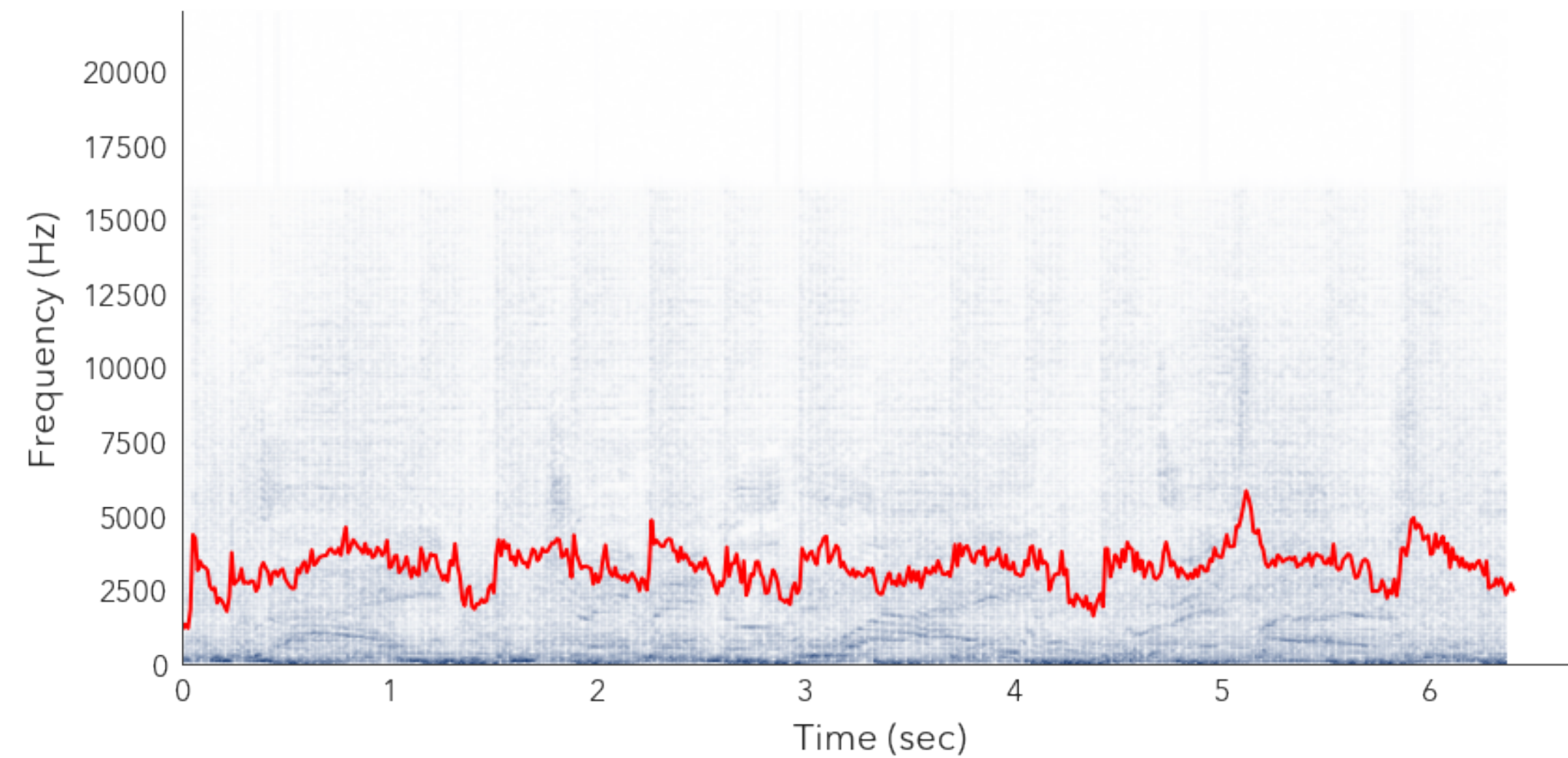
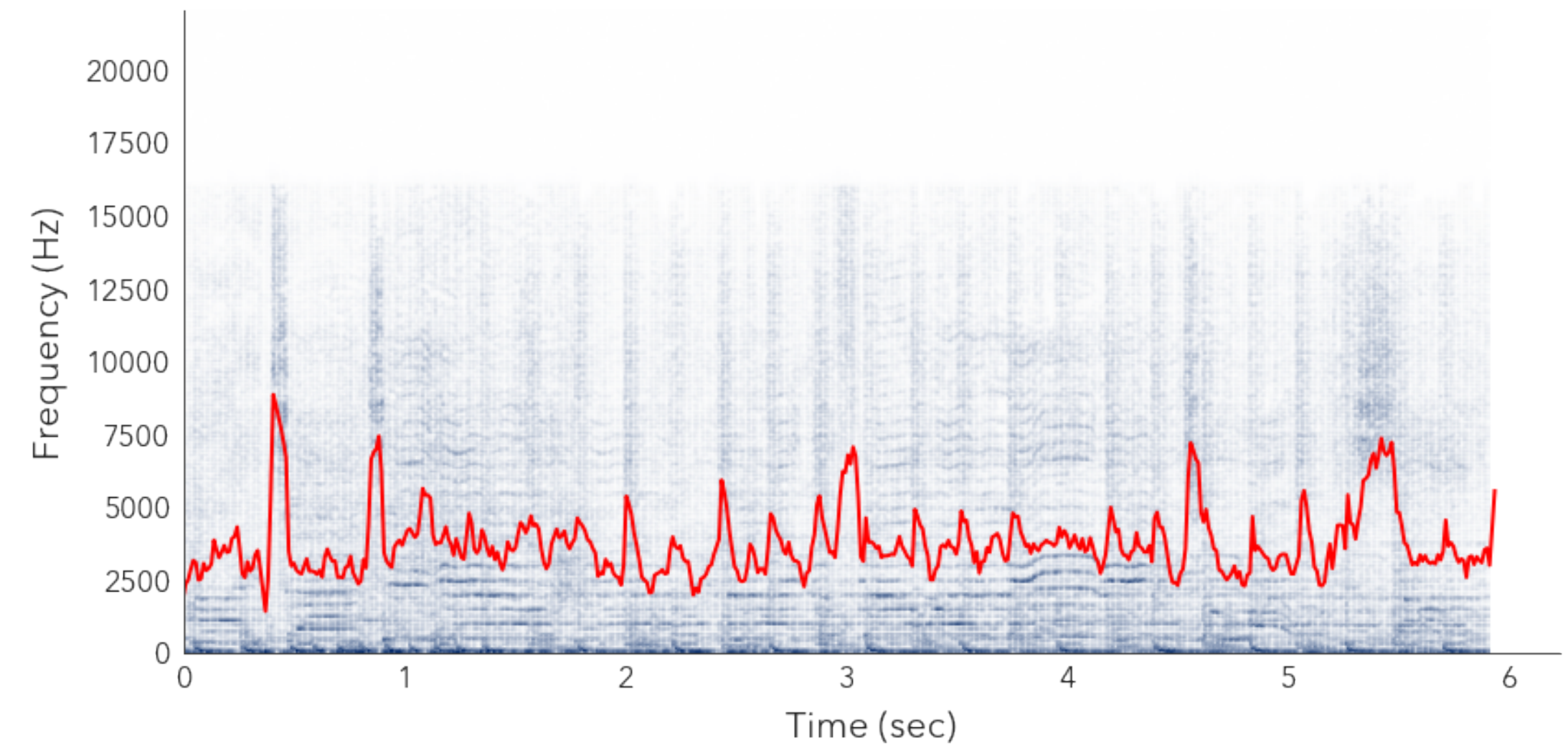
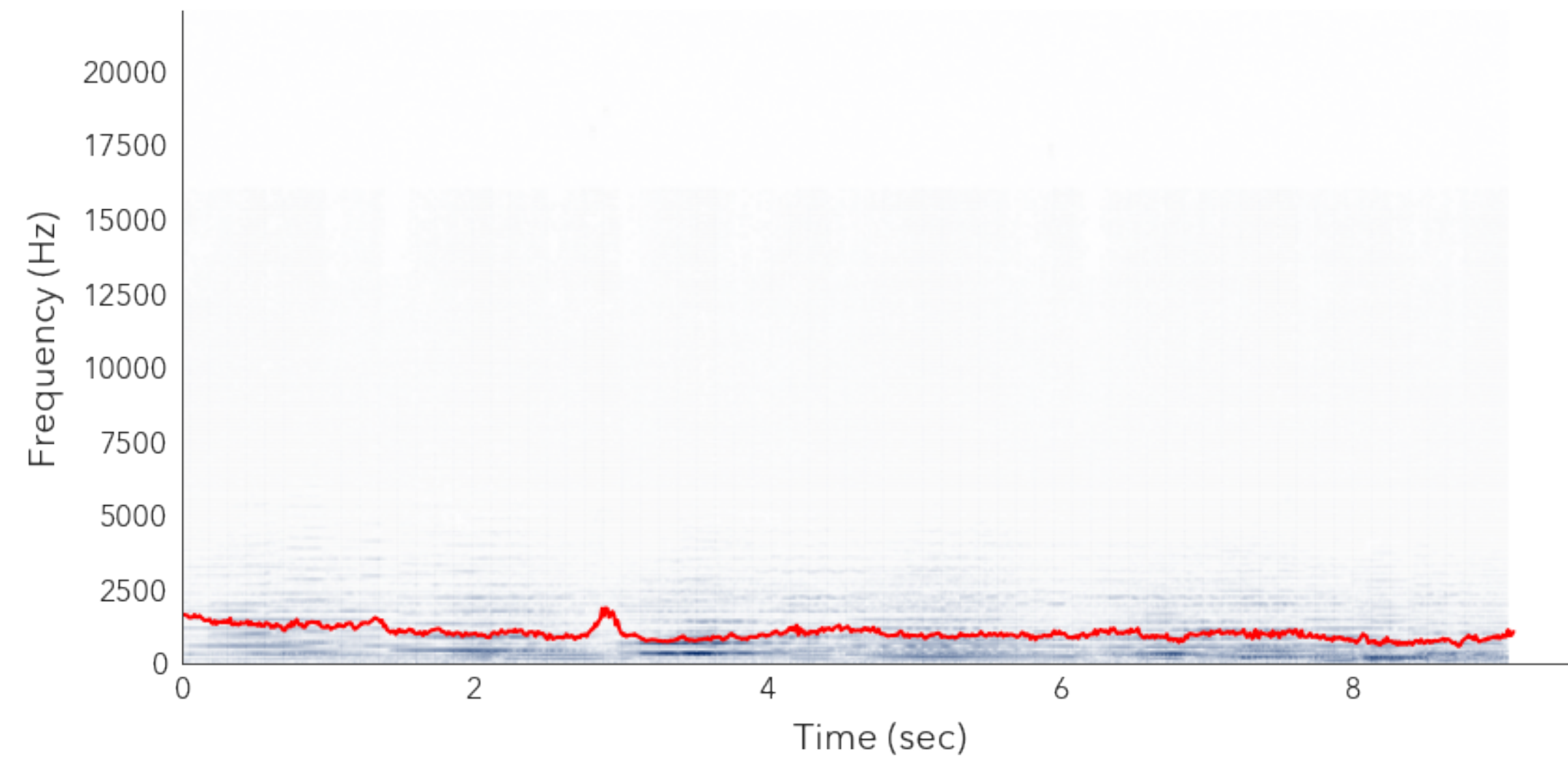
Spectral centroid

- “Center of gravity” of the spectrum

$$C_t = \frac{\sum \|F_t(f)\| f}{\sum \|F_t(f)\|}$$

- Correlates to perceived “brightness”

Spectral centroid



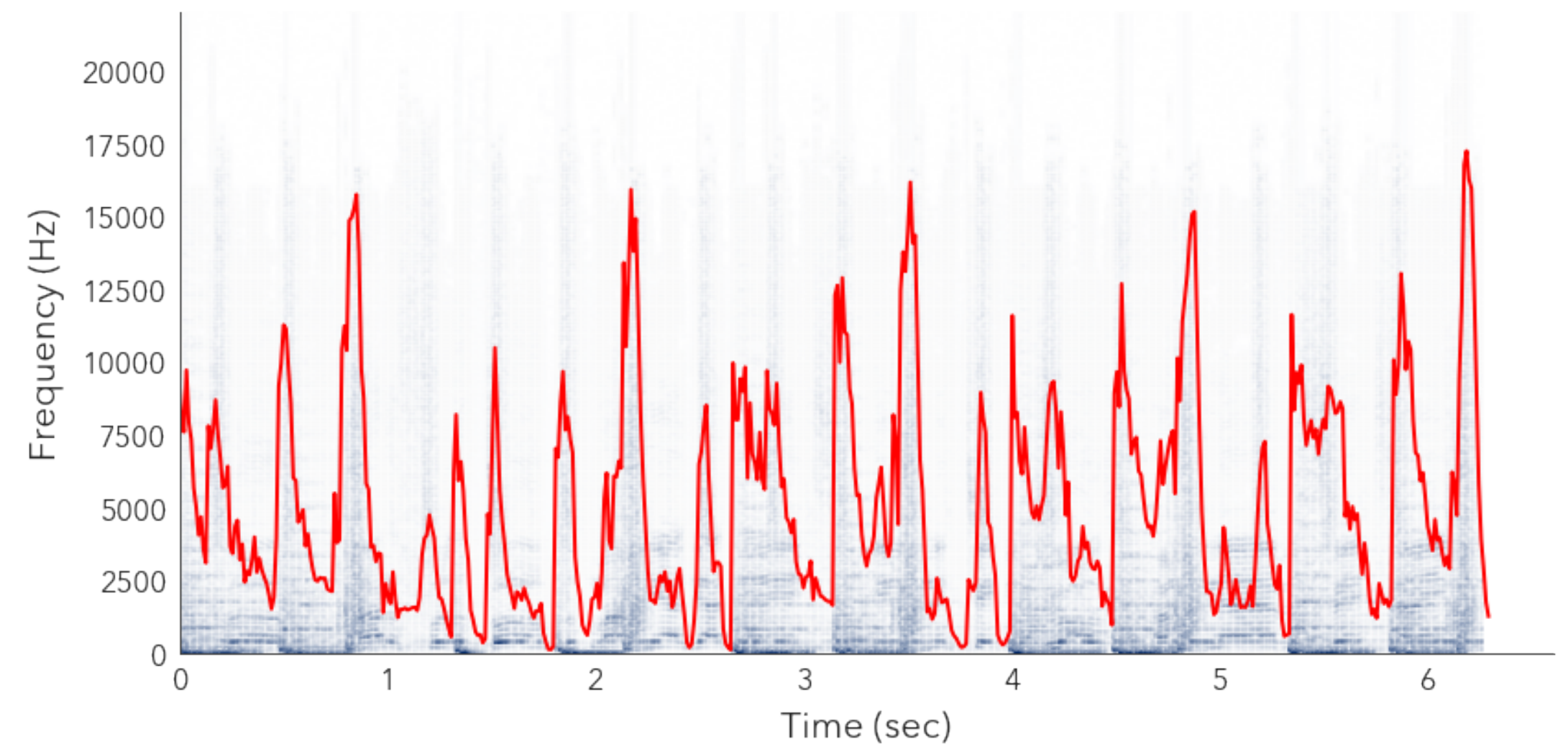
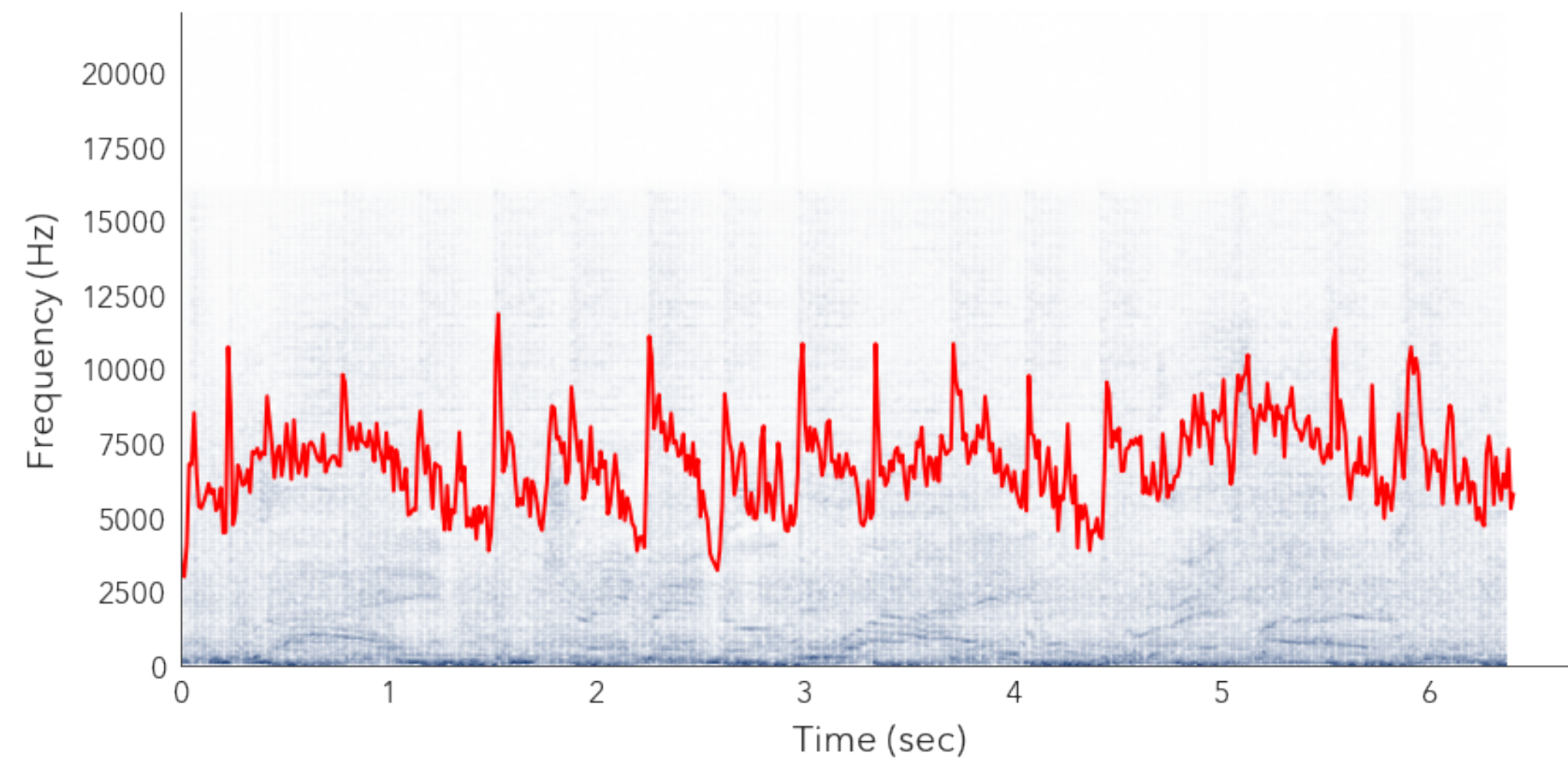
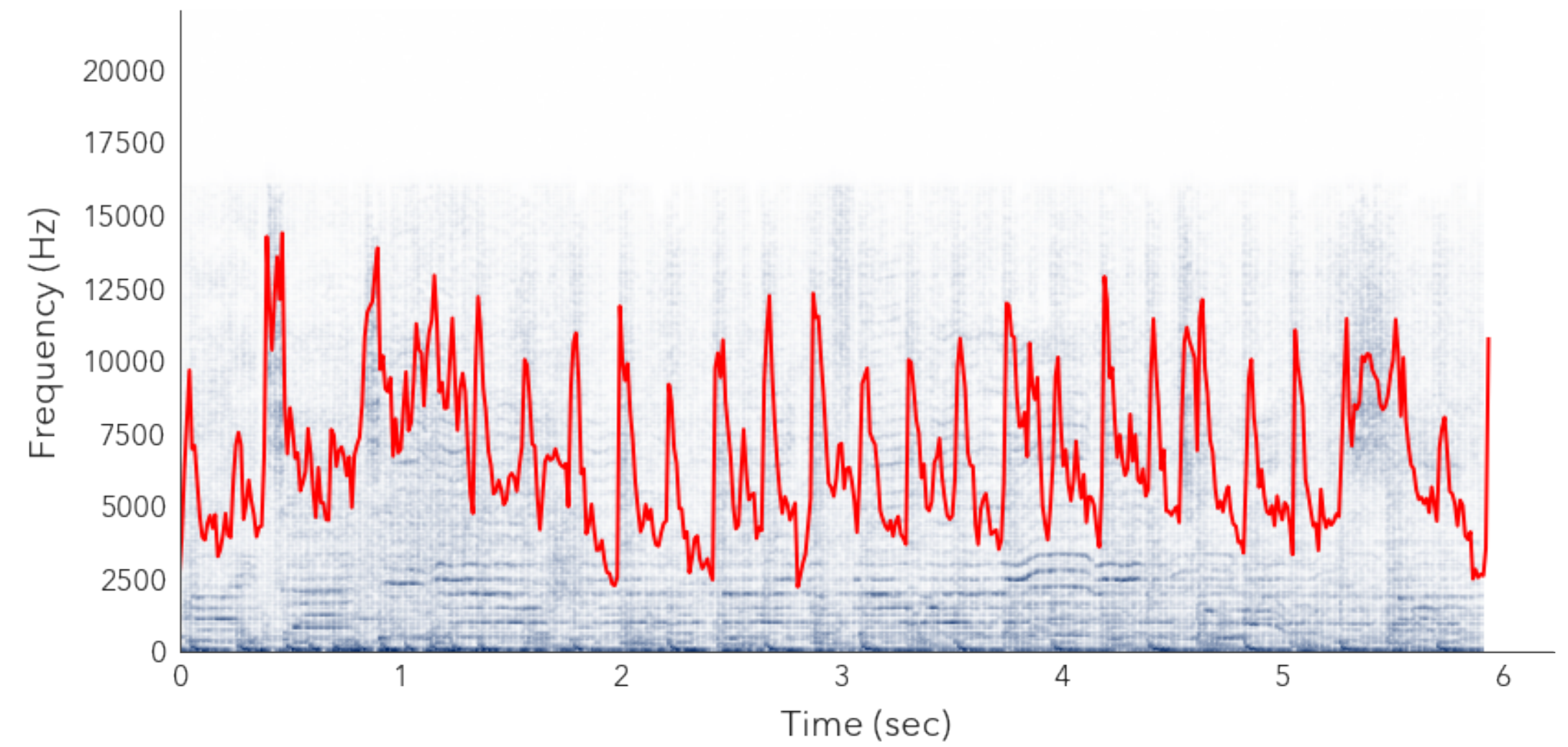
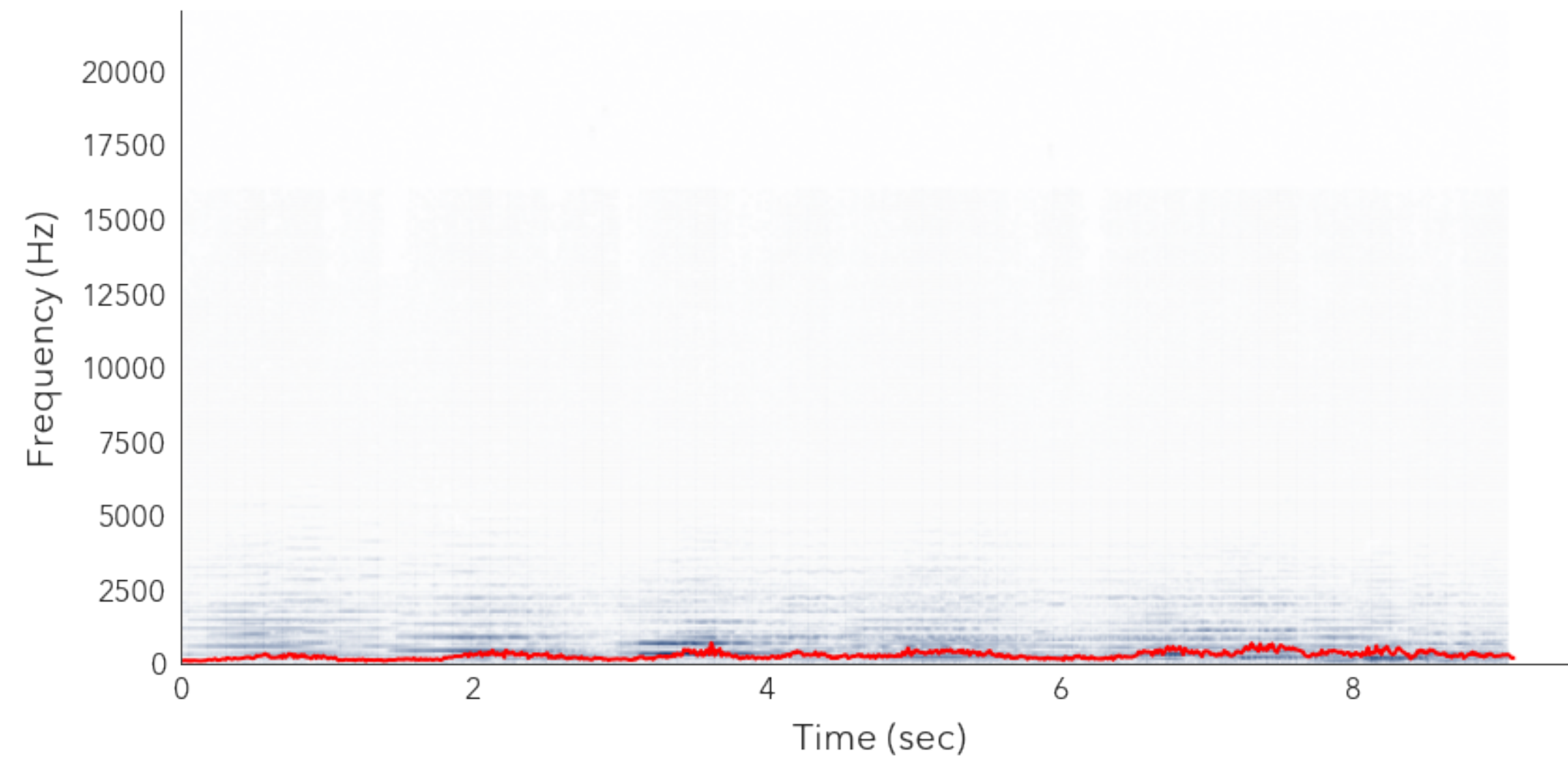
Spectral flux

- Difference between successive spectra

$$L_t = \sum [F_t(f) - F_{t-1}(f)]^2$$

- Provides a sense of local timbre change
 - Is the music static or dynamic?

Spectral flux



Many more ...

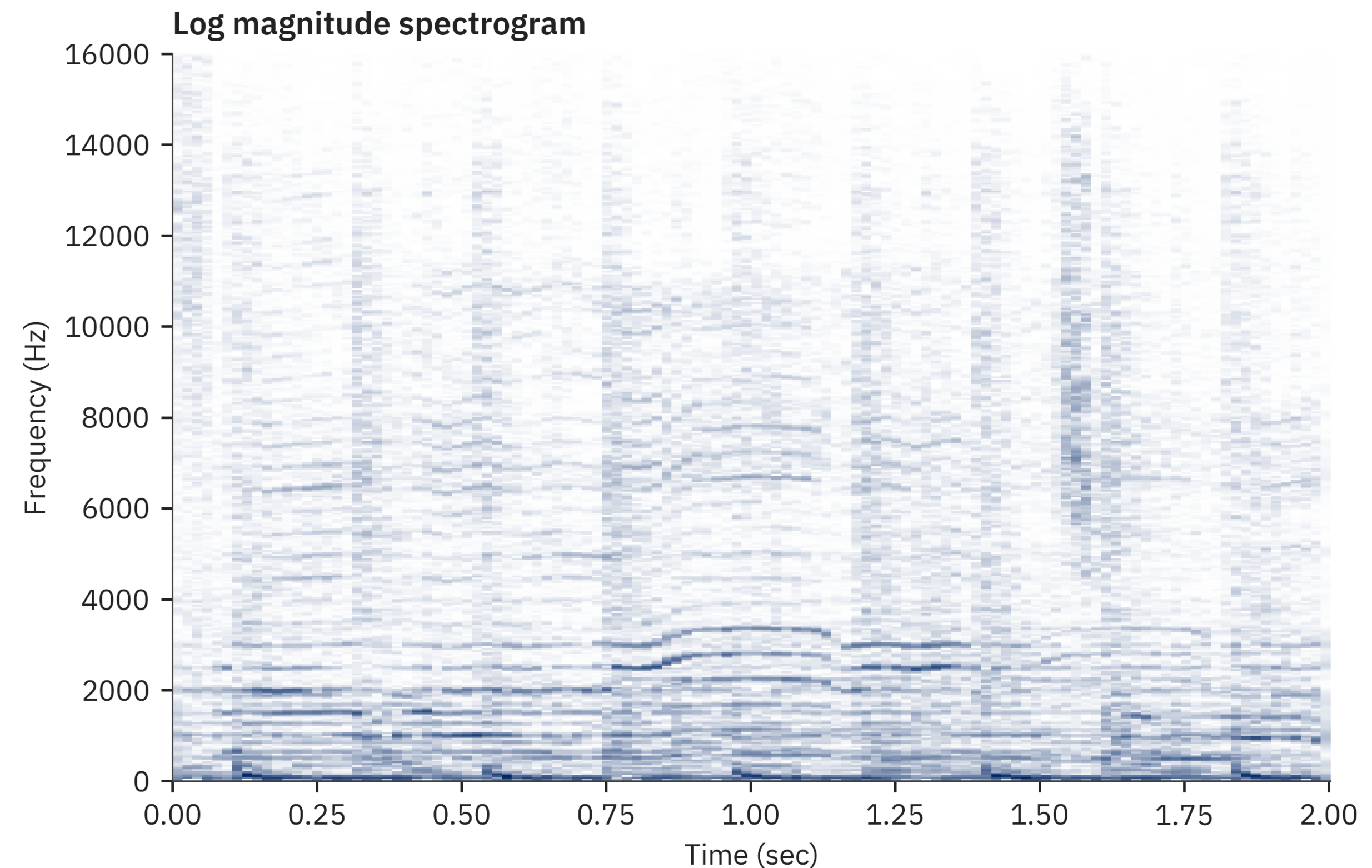
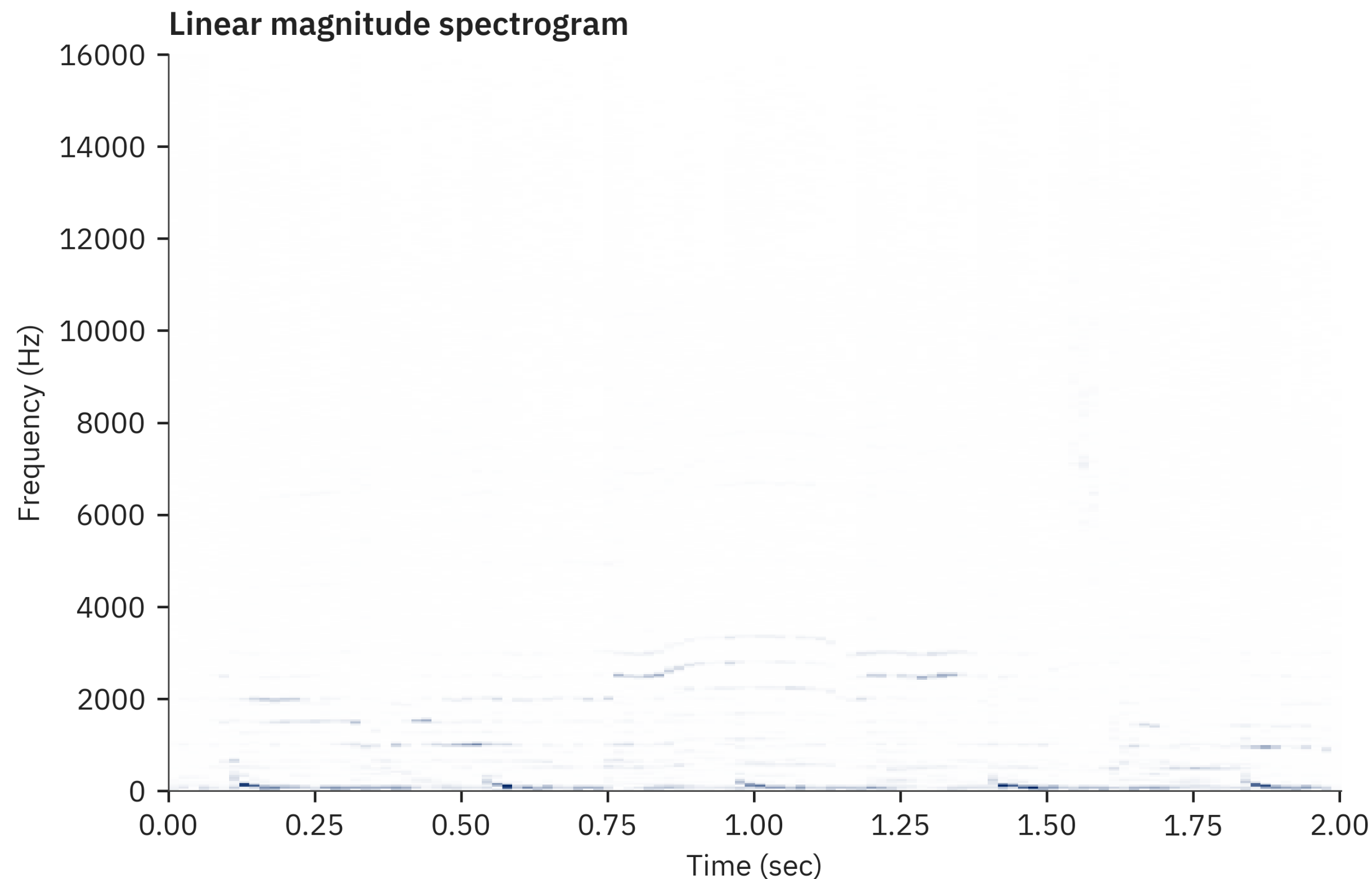
- Spectral slope
 - Fit a line to the spectrum slope
- Spectral statistics
 - Spread, skew, kurtosis
- Zero crossings per time window
 - Provides a sense of noisiness
- etc, ...

MFCCs

- *Mel Frequency Cepstral Coefficients*
 - A coarse version of the spectrogram
 - Blurring frequencies, removing redundancy
- Standard tool for speech, very popular for non-speech sounds as well

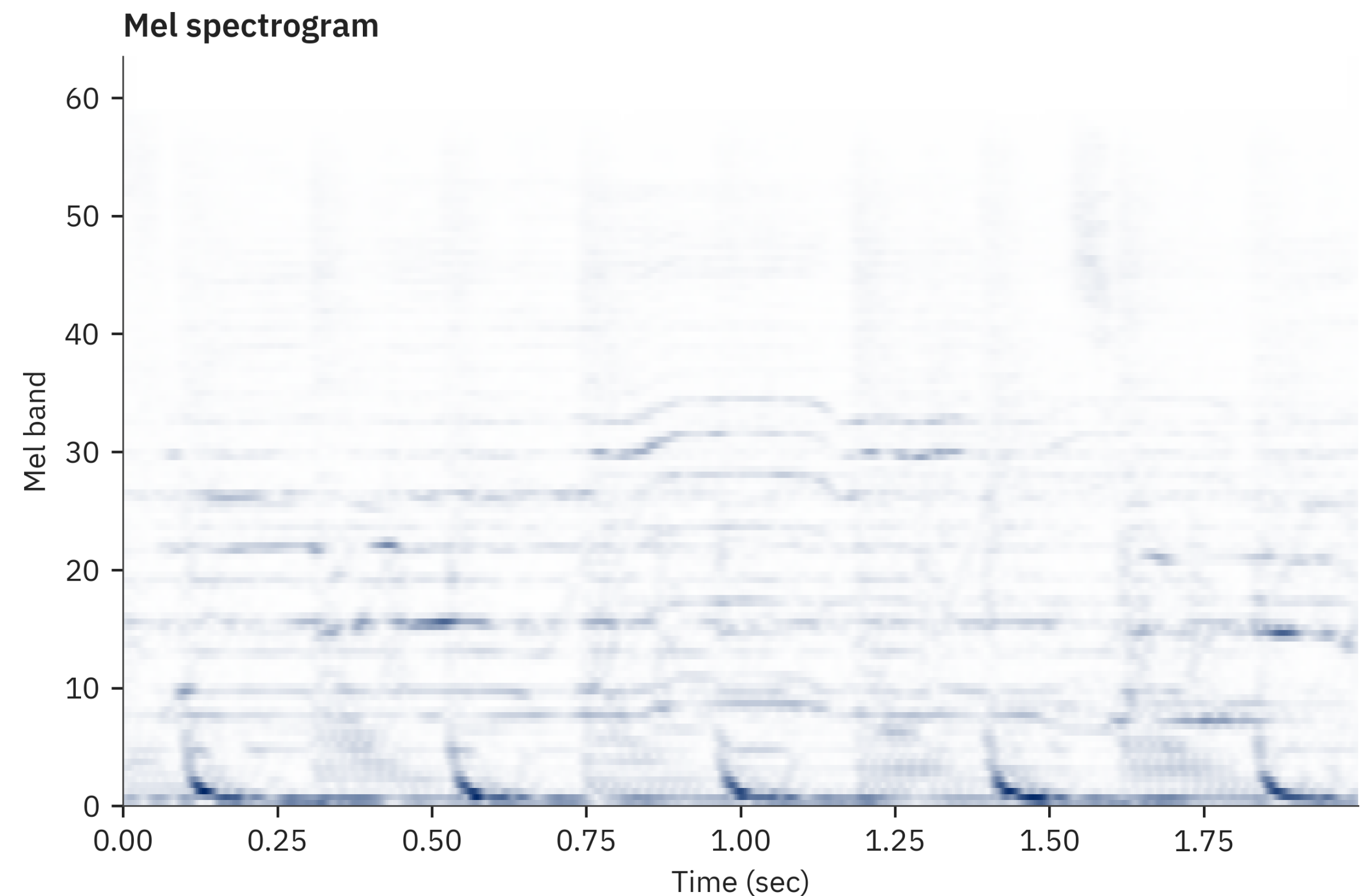
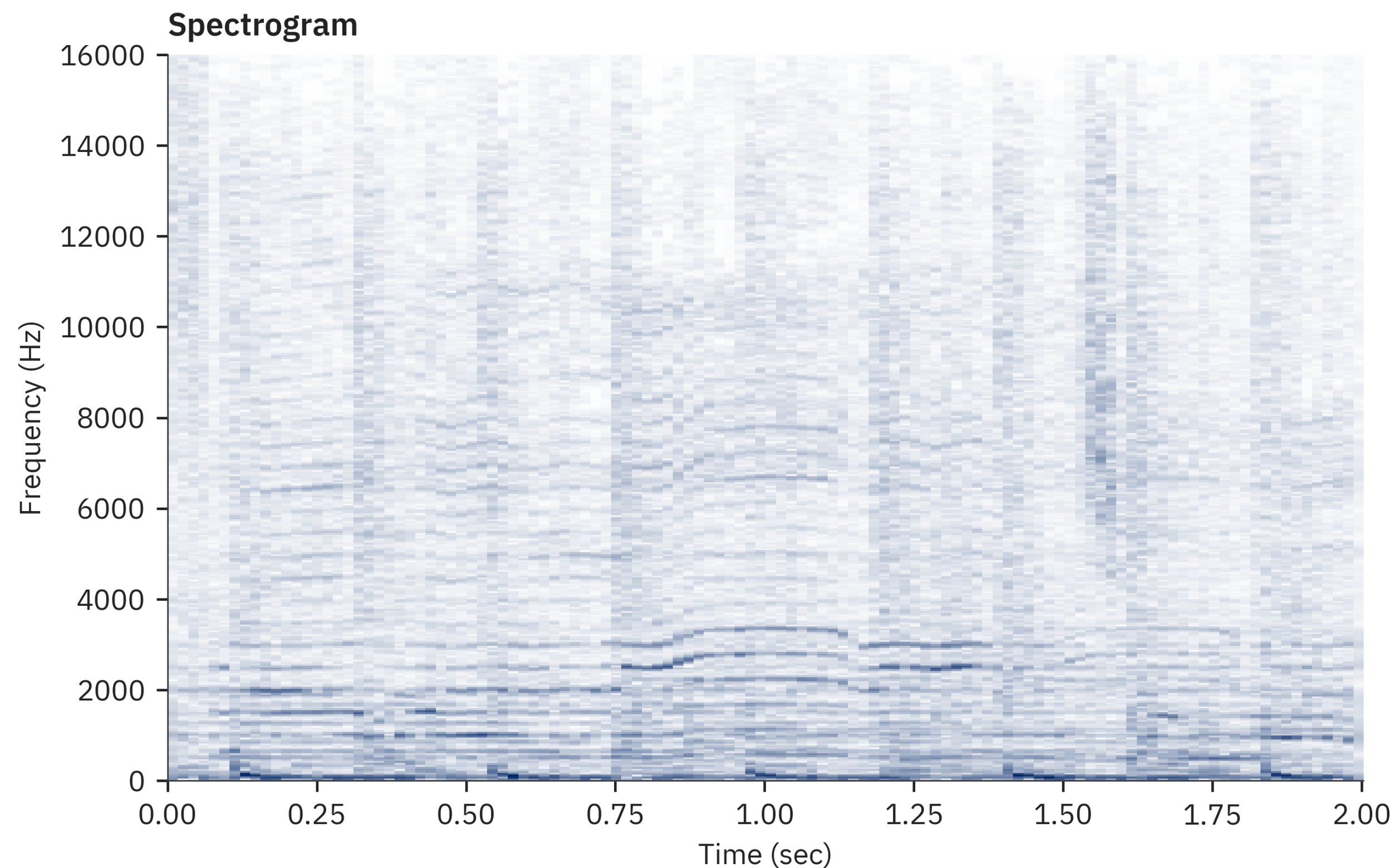
Making things more visible

- After the STFT we scale the magnitudes
 - Usually take their log, or raise them to a small power



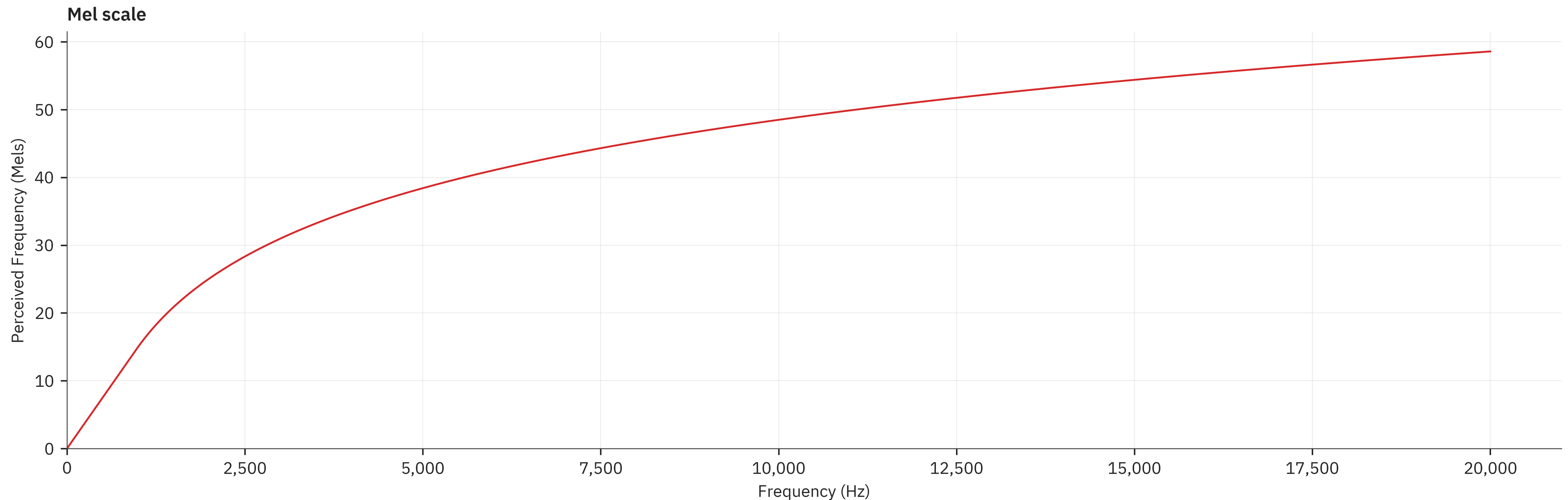
Frequency warping/rescaling

- Spectrograms are too detailed
 - Use *Mel* warping and reduce their number



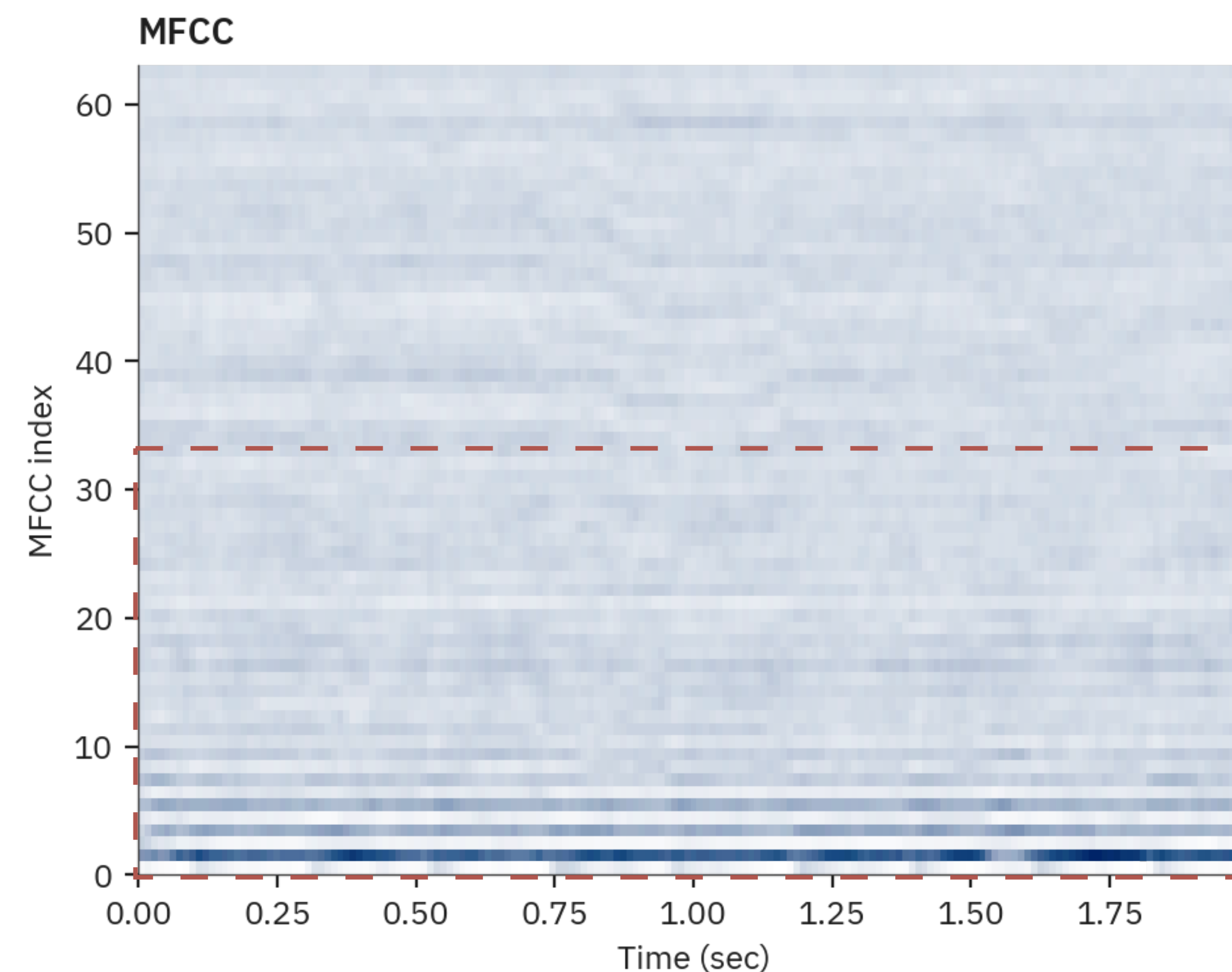
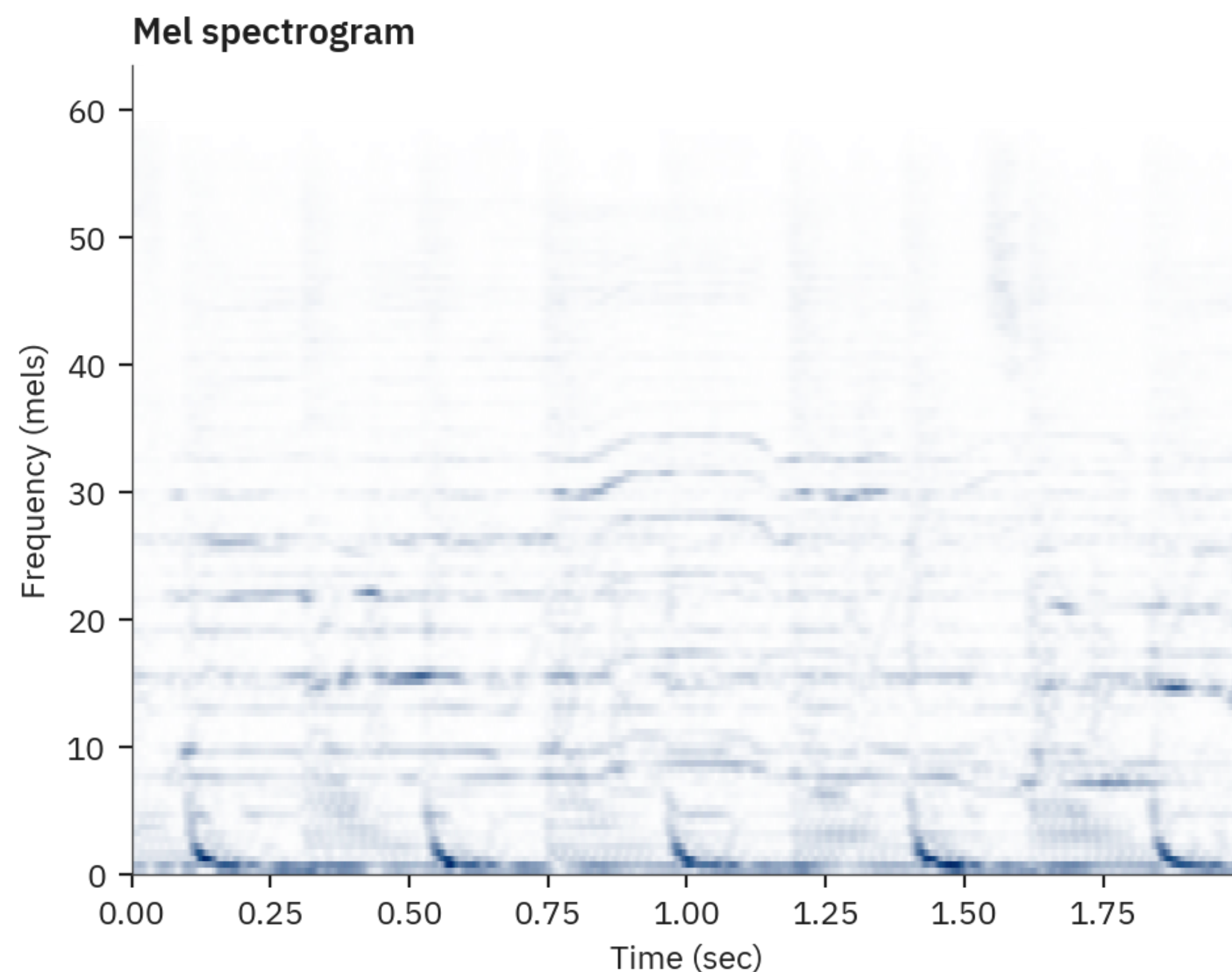
Mel what?

- Our perception of frequency is not linear
 - Mel scale maps Hz to a perceptual scale



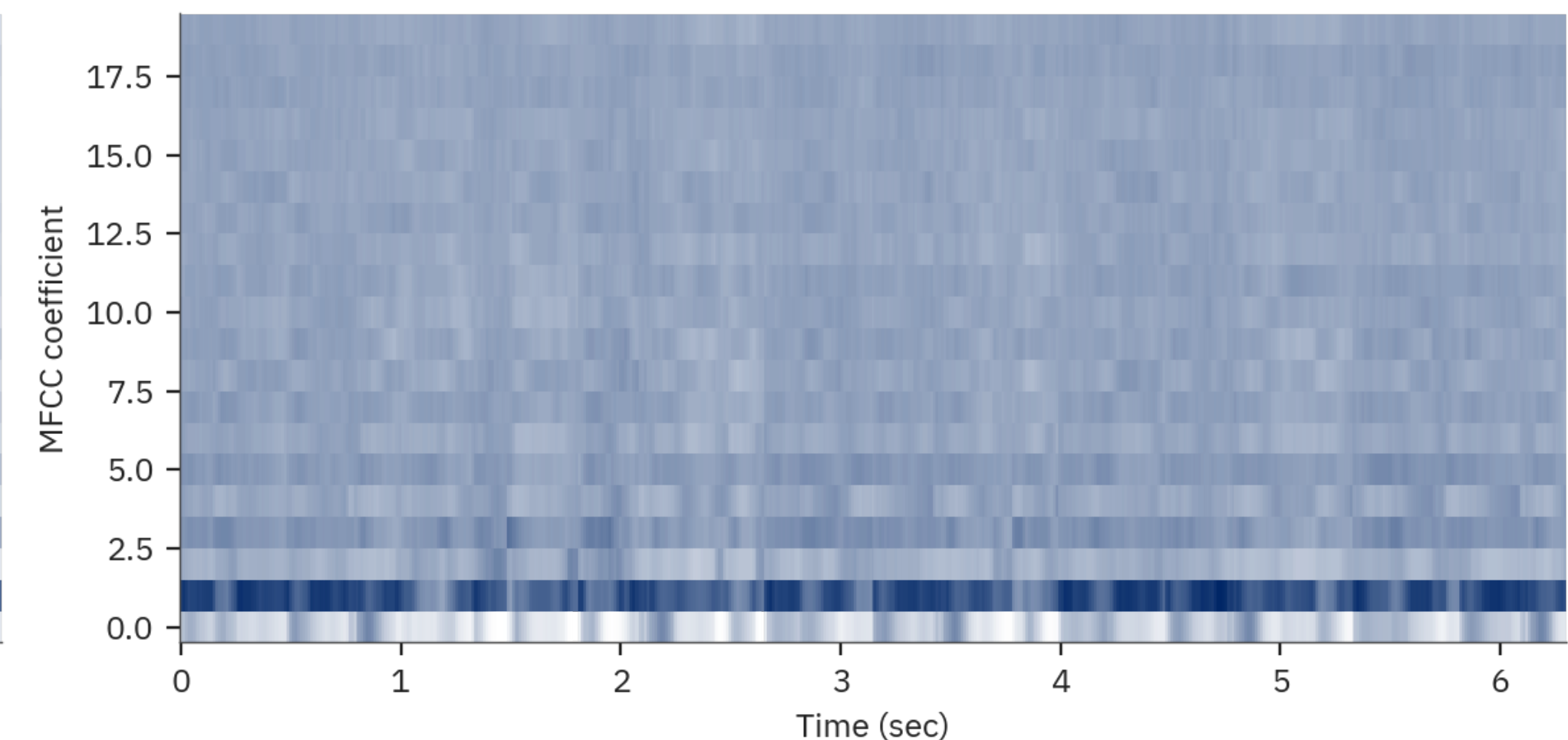
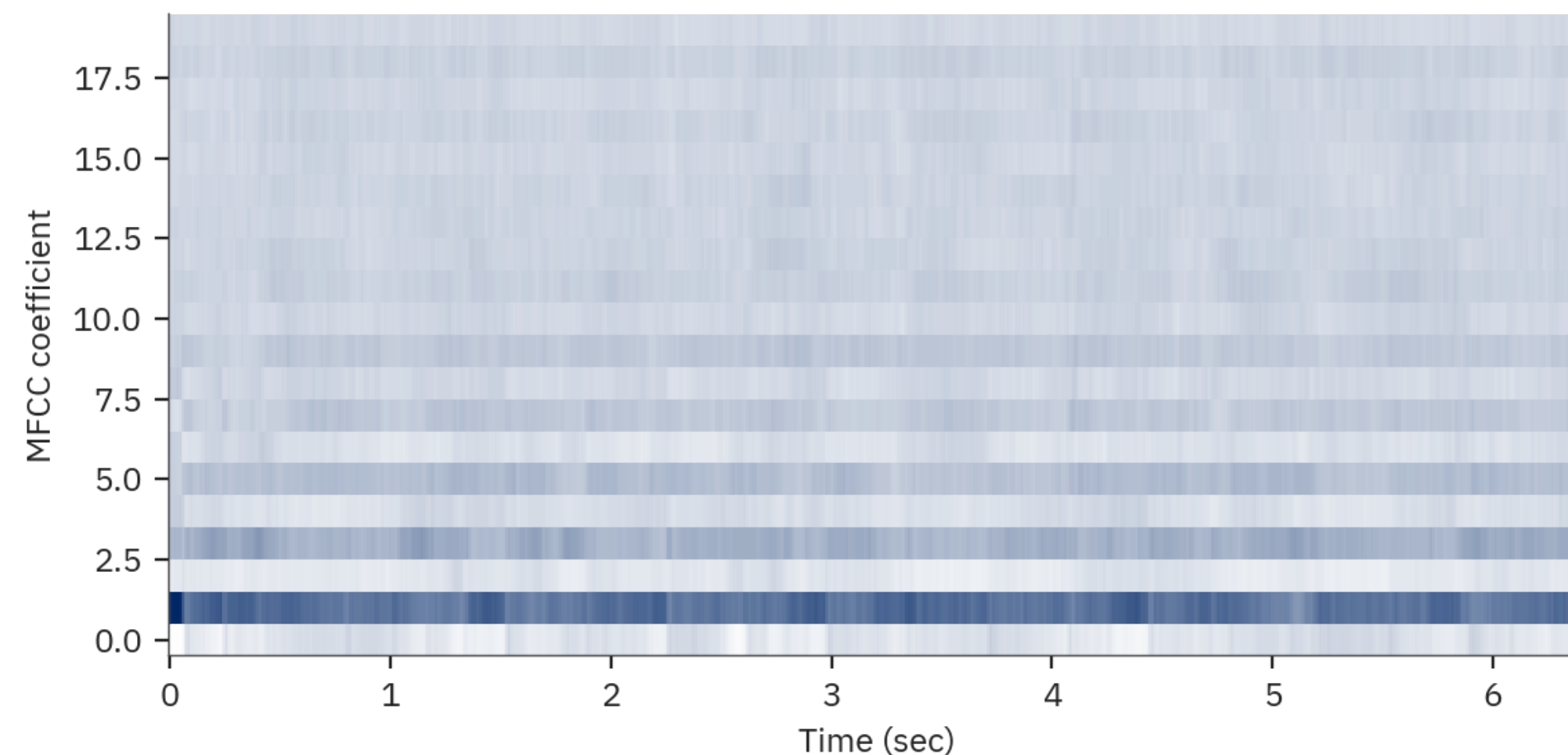
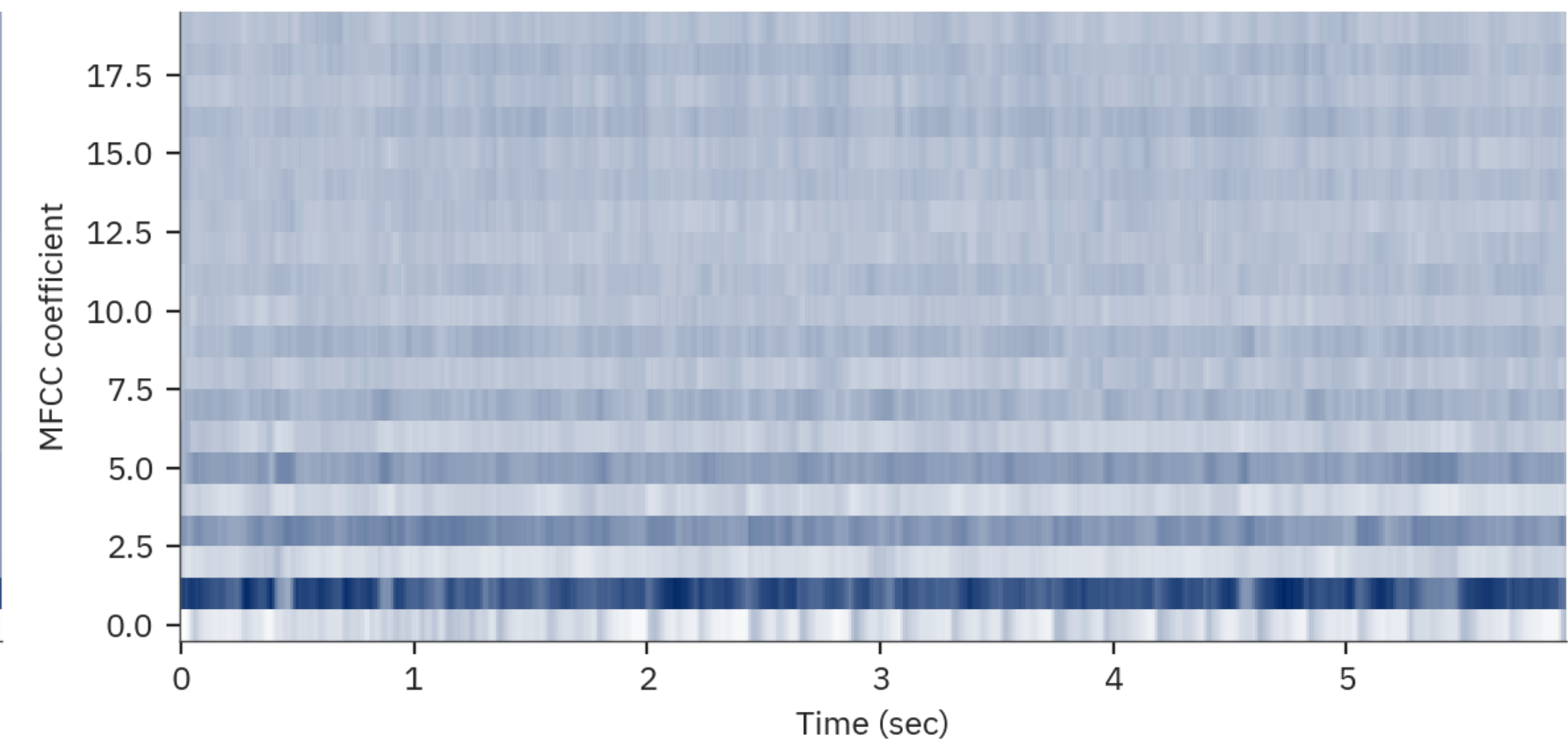
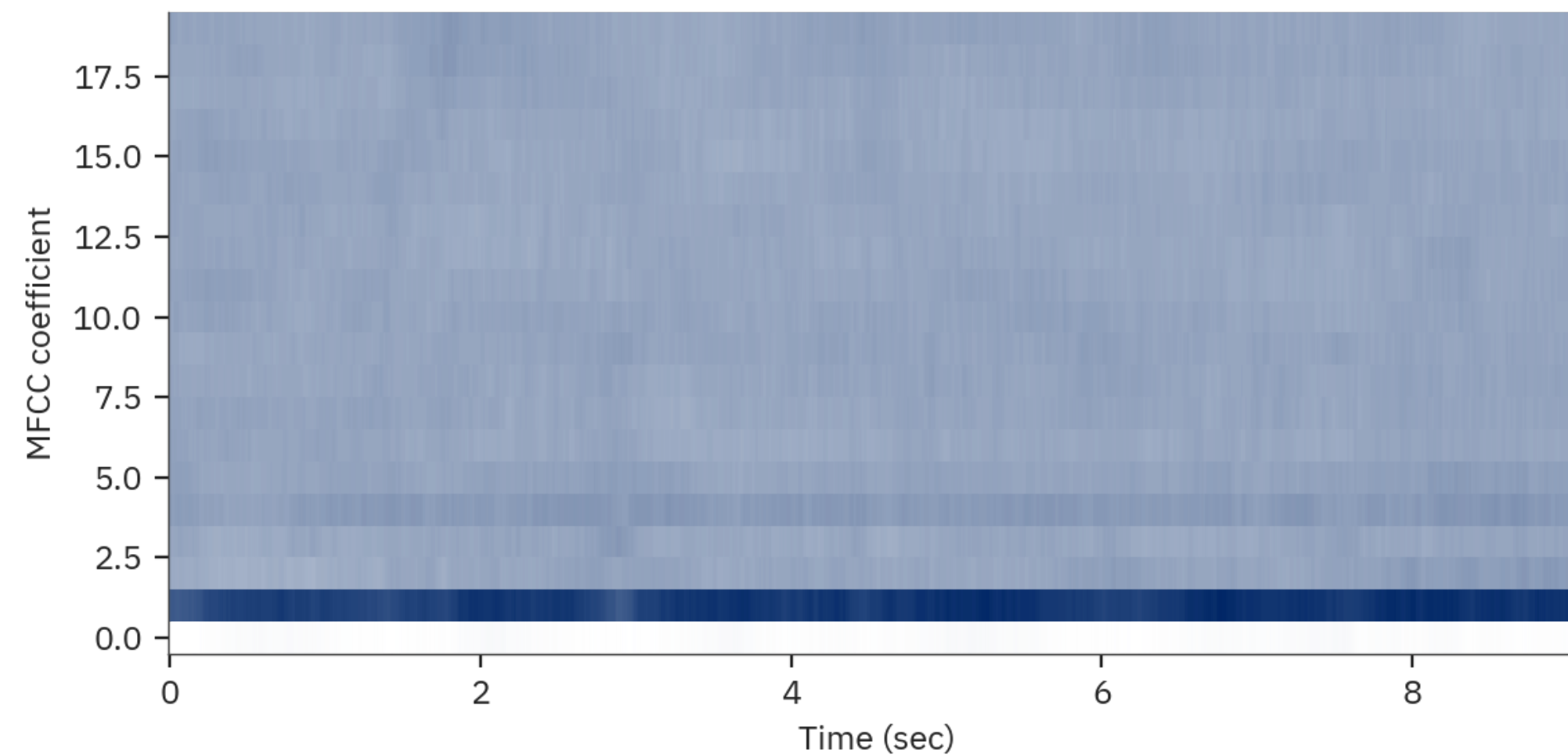
Dimensionality reduction

- Do a DCT (Discrete Cosine Transform)
 - Keep lowest frequencies



Resulting transform

- Not as visually intuitive, but good for classification



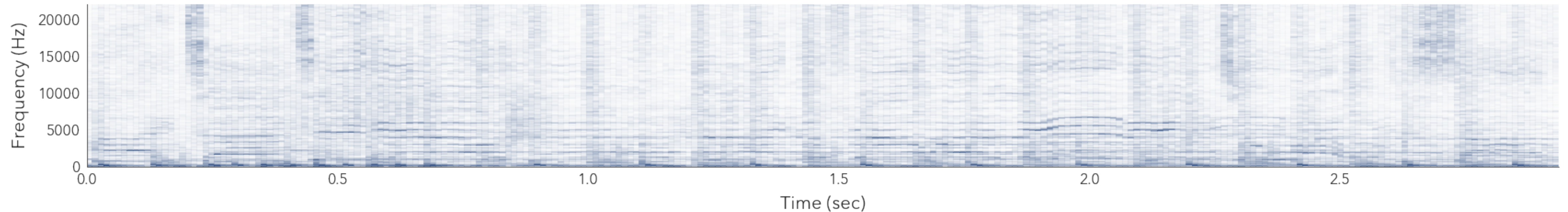
Why did we do all that?

- Magnitude log scaling
 - Magnitude spectra are exponentially distributed
 - Taking their log makes them more Gaussian distributed
 - So we can confidently use our classifiers
- Mel warping
 - Warps the frequencies to map to our perceptual space
- DCT
 - We only keep the smooth spectral movements

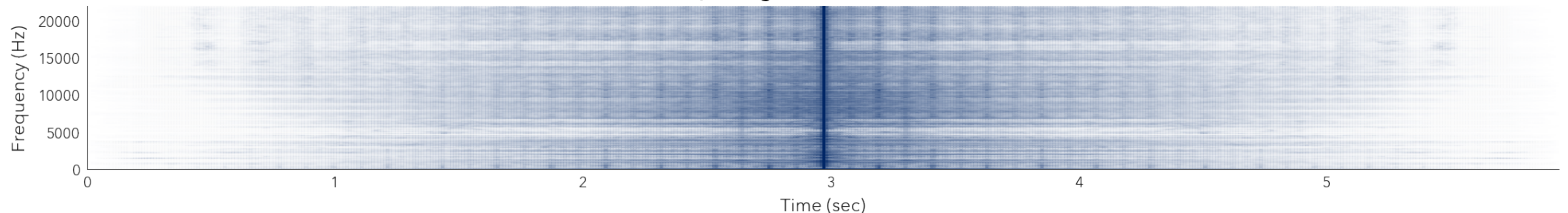
Rhythm features

- Get temporal correlation
 - Autocorrelate each frequency band
 - Reveals period of repetition → tempo

Spectrogram



Spectrogram autocorrelation



Putting it all together

- Append all of the features in a vector
 - e.g. $\mathbf{x} = [mfcc, centroid, flux, \dots]$
- Use that representation for classification
 - Same thing as we did previously
- Can work quite ok
 - 60%-70% accuracy in reasonably realistic data sets

Reusing this framework

- Artist recognition
 - Madonna sounds different from Metallica
- Music similarity
 - Measure similarity between two recordings
- Music clustering
 - Group large music collections based on acoustical similarity

Audio fingerprinting

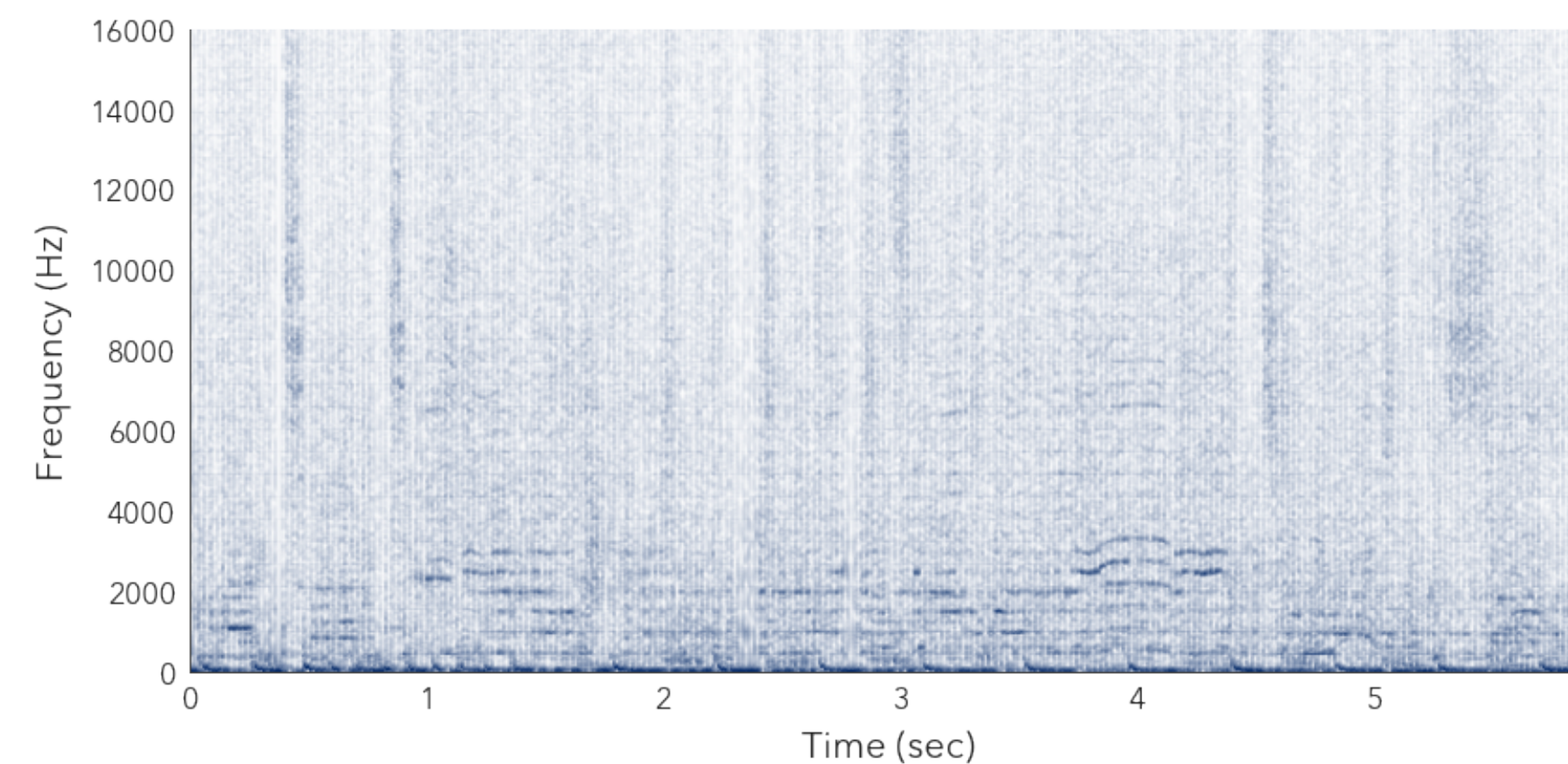
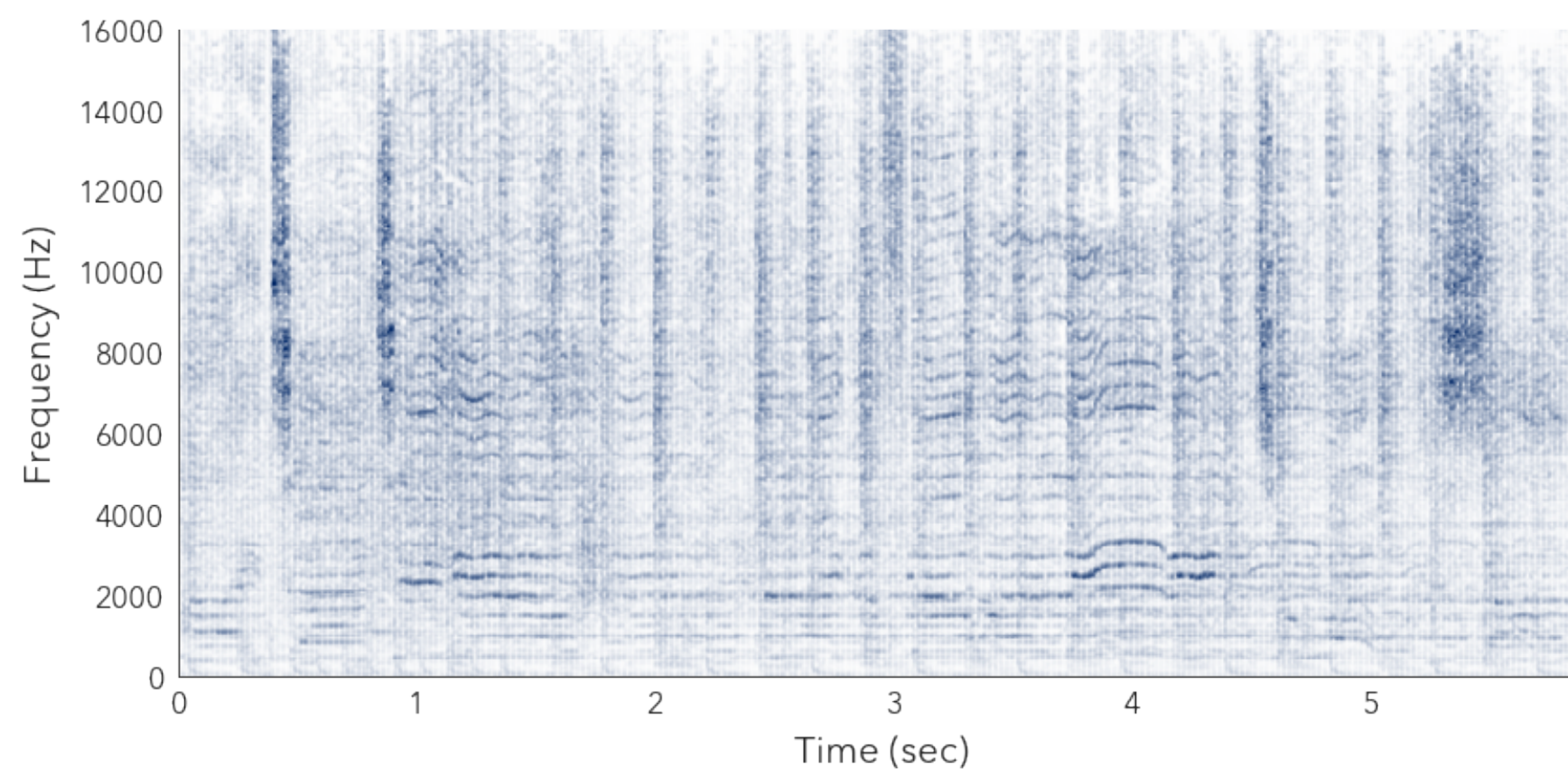
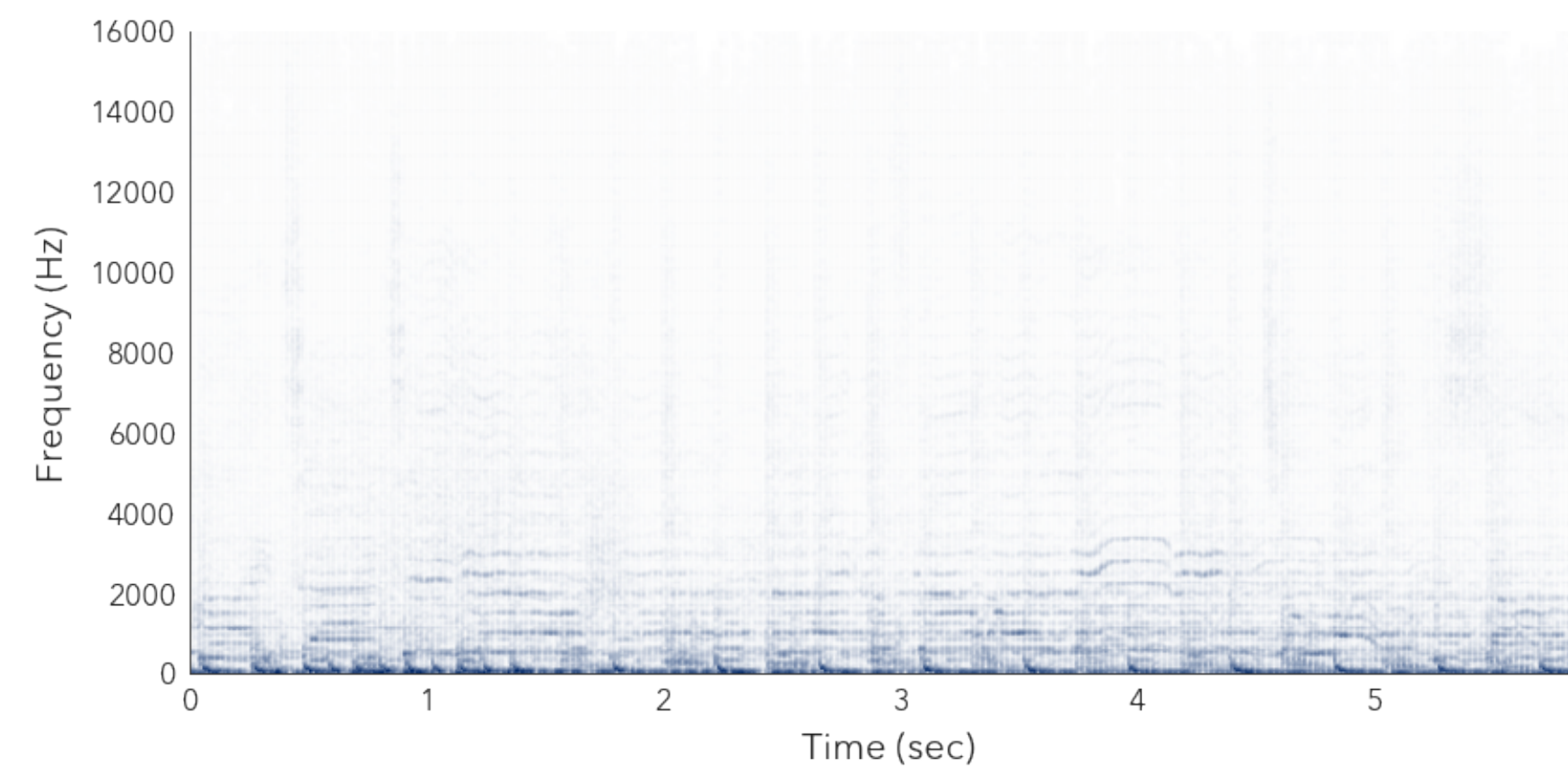
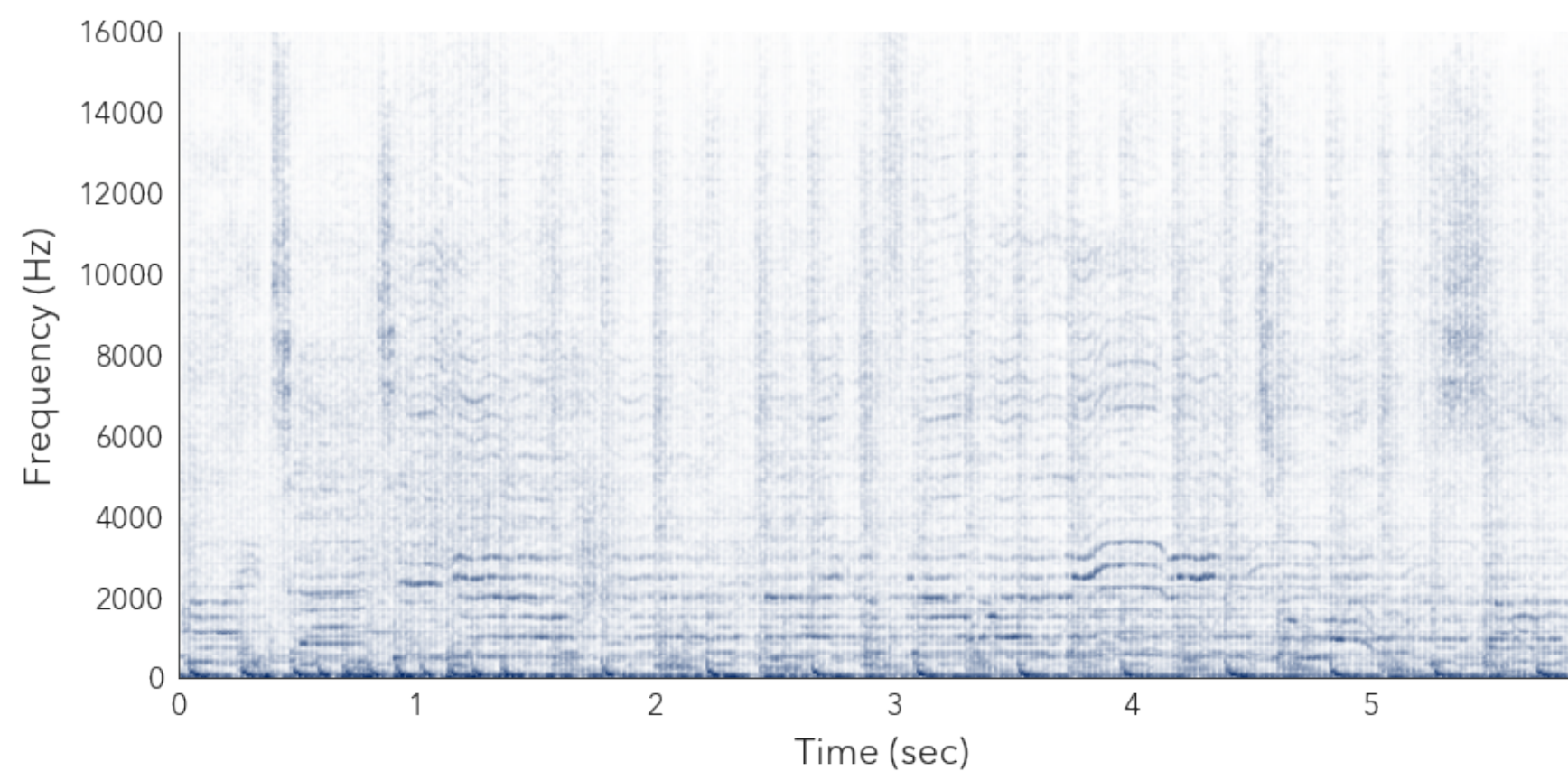
- So far we looked at approximate matches
- Fingerprinting finds exact matches
 - From potentially noisy recordings
- Is this song *X* or not?
 - Widely available commercially, e.g. Shazam

How do we do the matching?

- Should be simple
 - Measure similarity of two recordings
 - e.g. same time samples, or same spectrograms, ...
 - Will this work?
- Problems
 - What if we have a noisy query?
 - How do you check over your library of millions of songs?

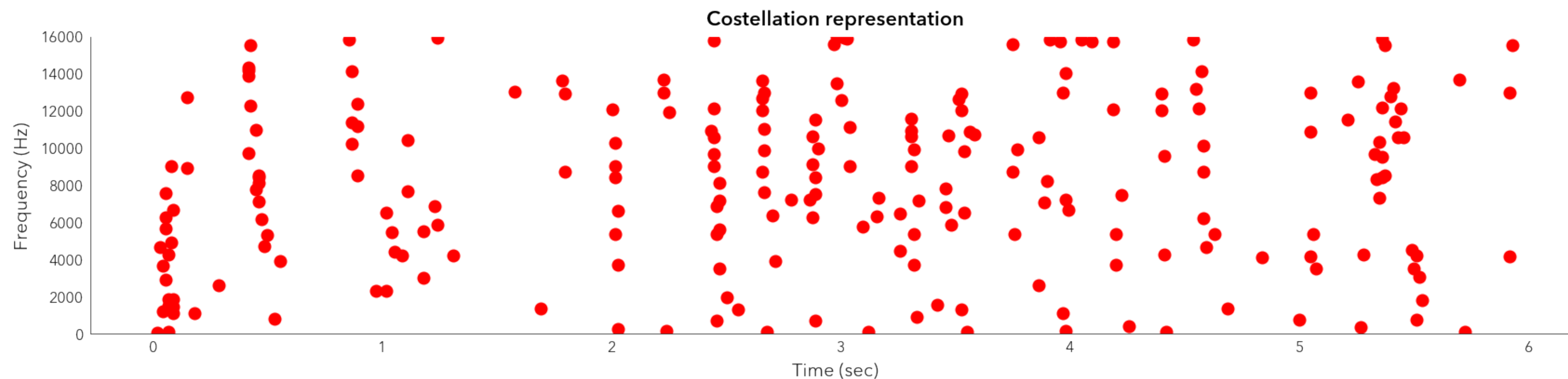
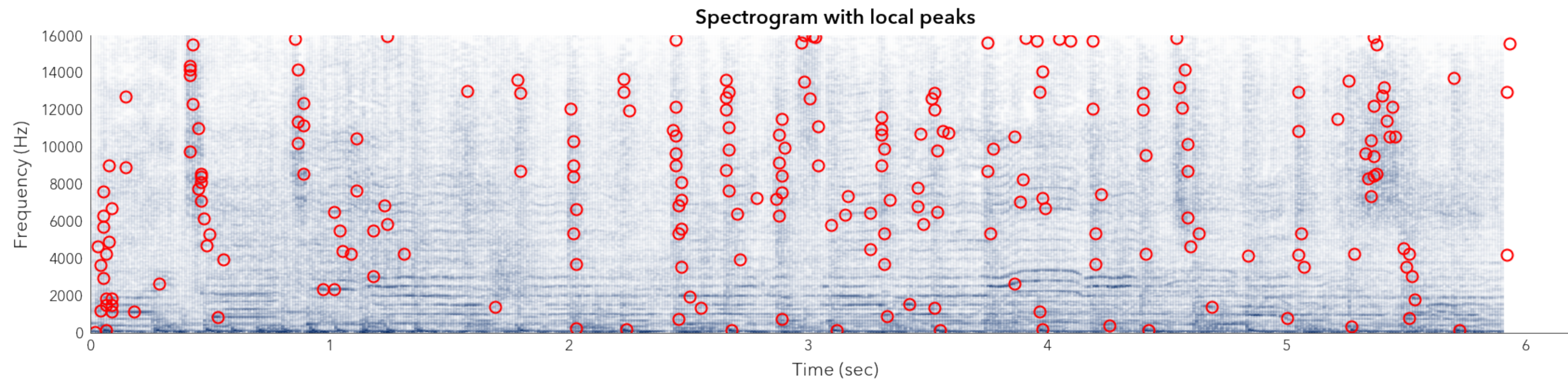
The need for robust features

- The query can be noisy or incomplete
 - Our features must not be swayed by that



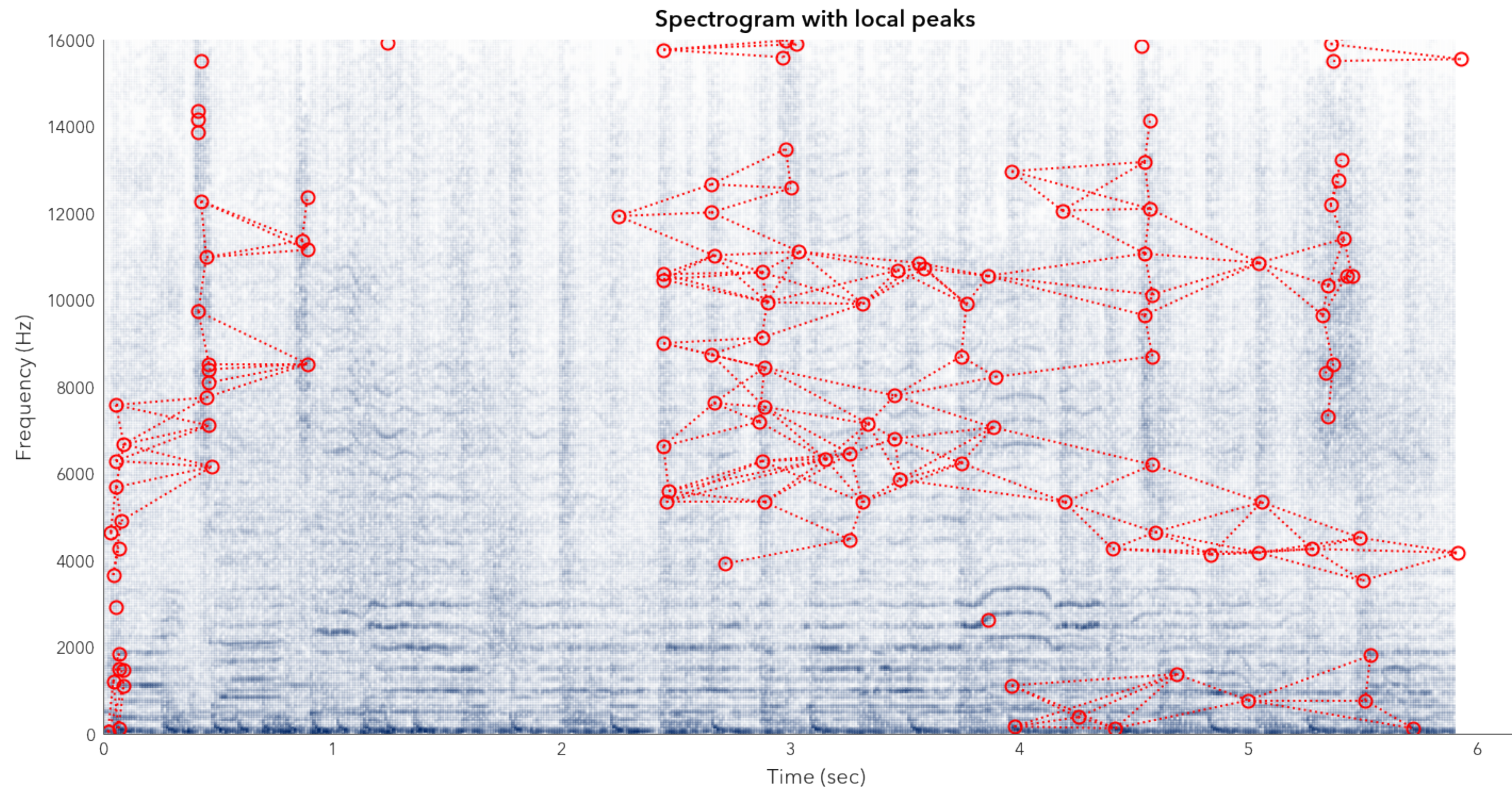
Finding salient peaks

- Find local peaks and note their positions
 - Spectrogram array is now represented by a set of points



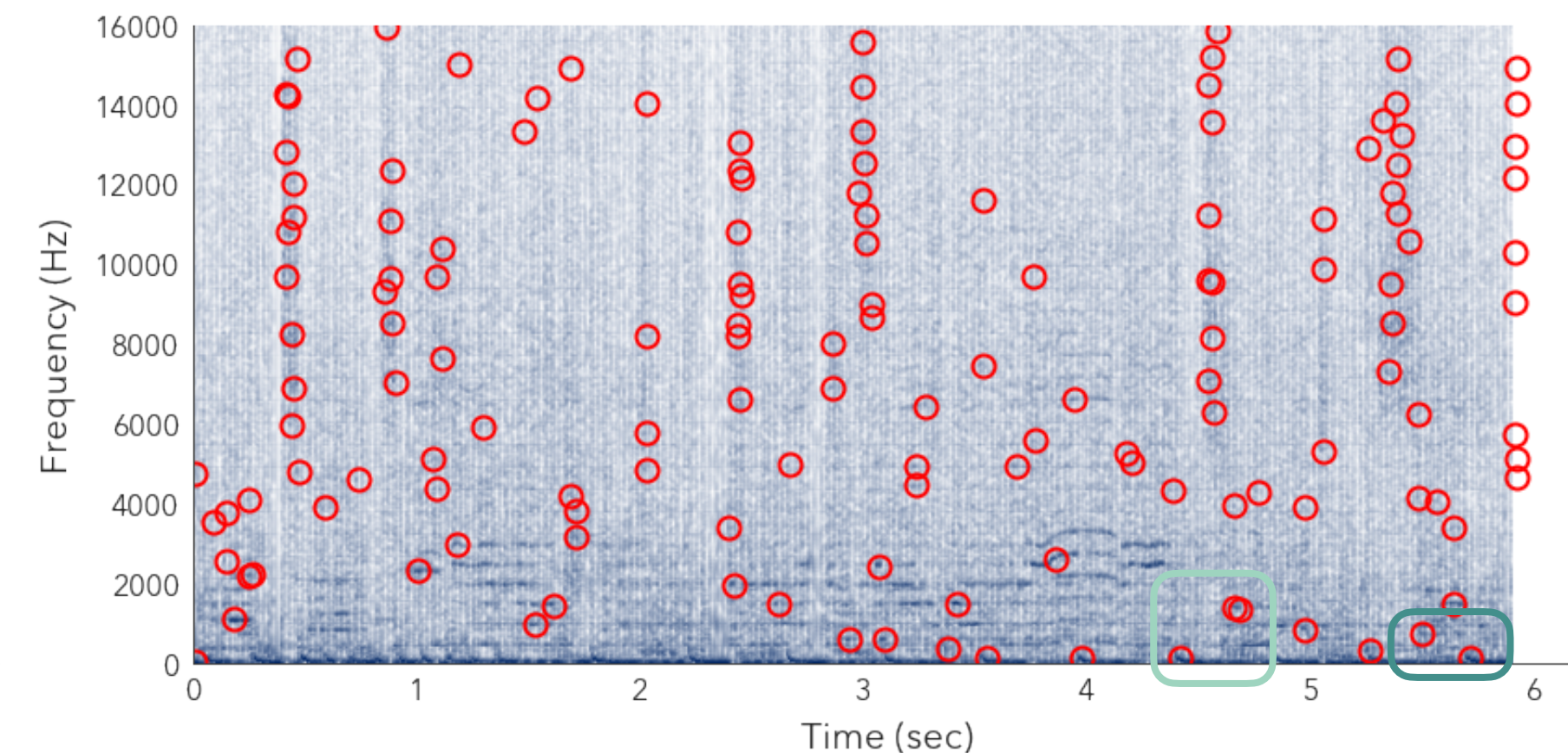
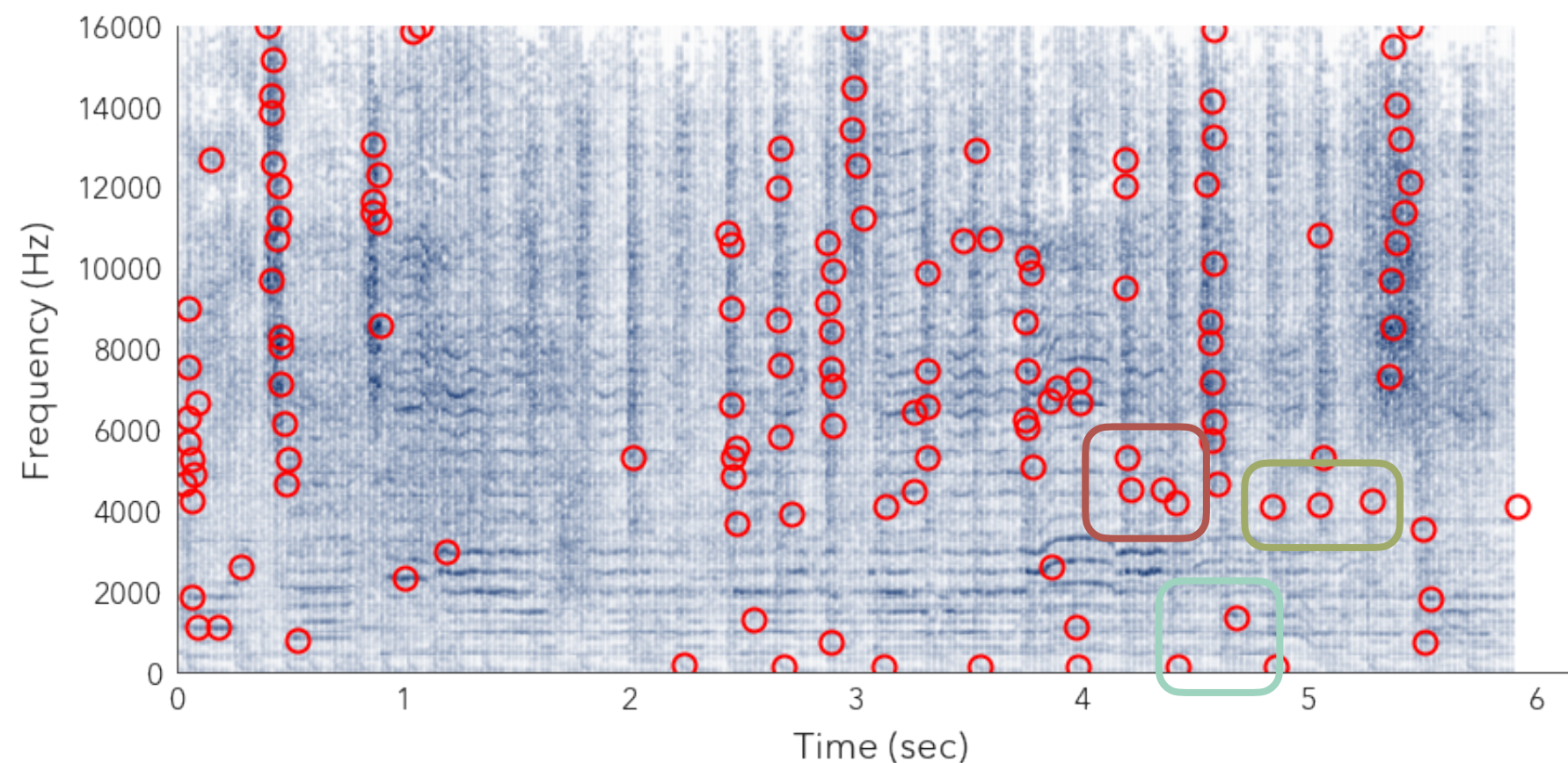
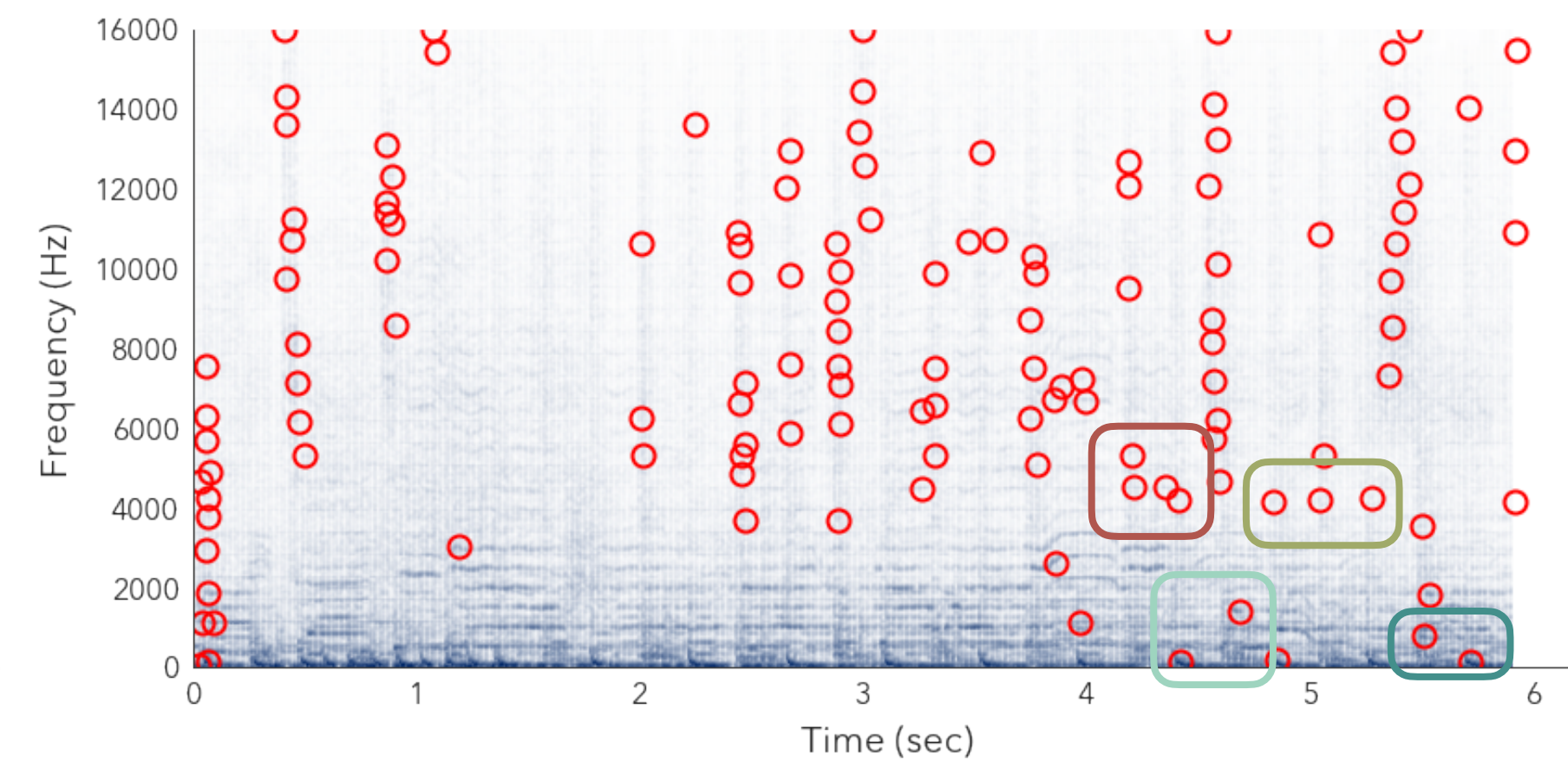
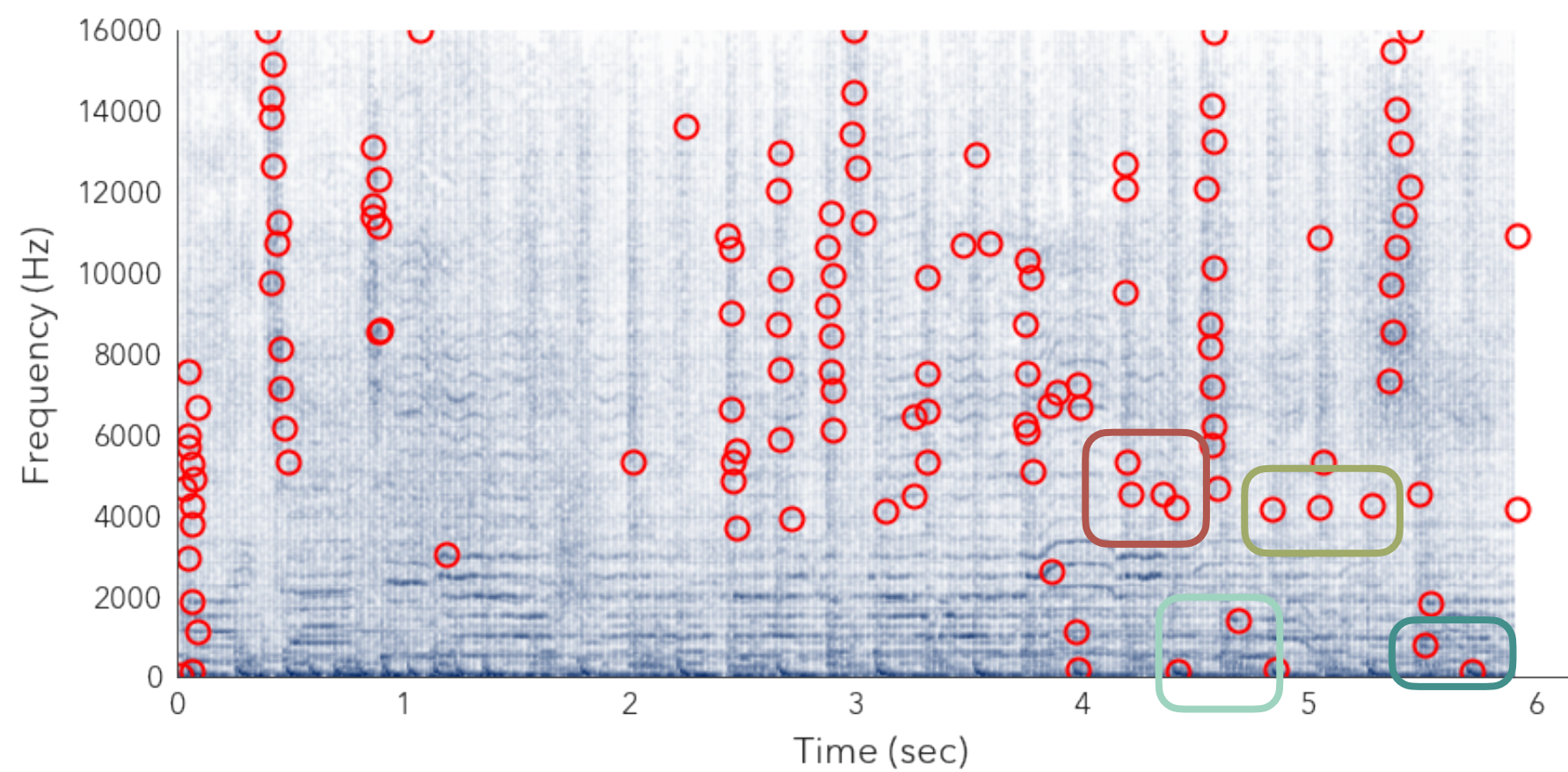
Forming constellations

- Connect neighboring peaks to form pairs
 - Information we keep: $time_1$ and $[time_2 - time_1, freq_1, freq_2]$



The advantage of constellations

- Constellations are rather robust
 - They do not change as drastically under noise, filtering, etc.

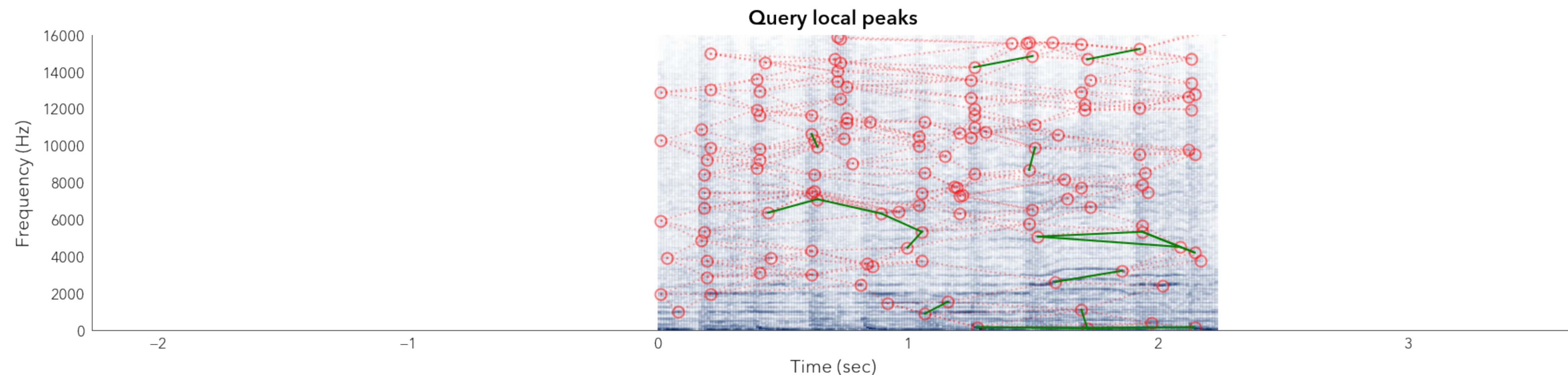
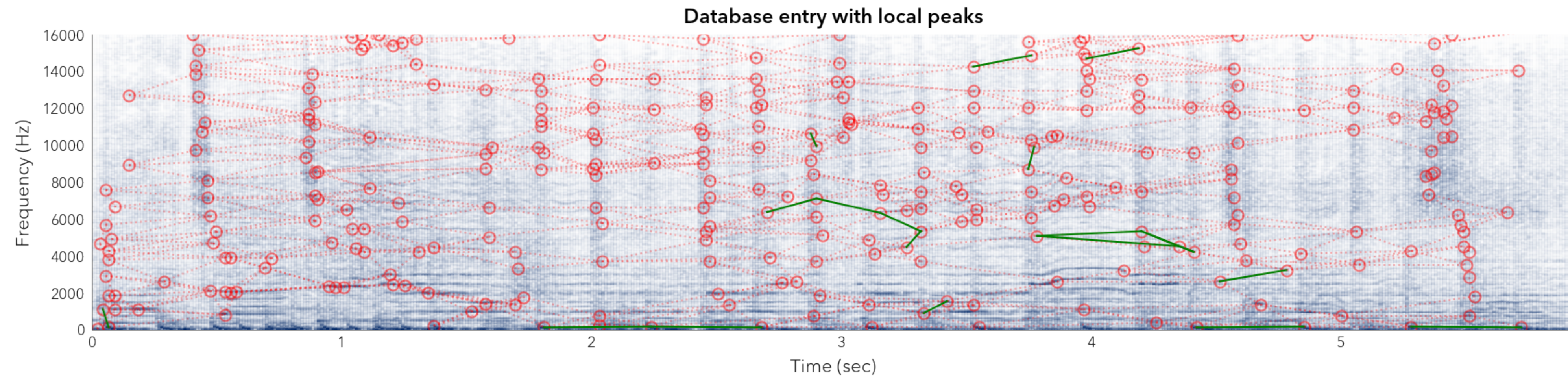


Hashing for fast searching

- For each recording make a table of constellations
 - Take all constellation features and hash them
 - e.g. pack in a 12 bit int: $\left[\underbrace{\text{xxxx}}_{dtime} \underbrace{\text{xxxx}}_{freq_1} \underbrace{\text{xxxx}}_{freq_2} \right]$
- Find database entries with common hashes to query
 - Common hashes imply similar spectral features
 - Lots of these imply a very similar spectrogram
- Very efficient since we can precompute database hashes

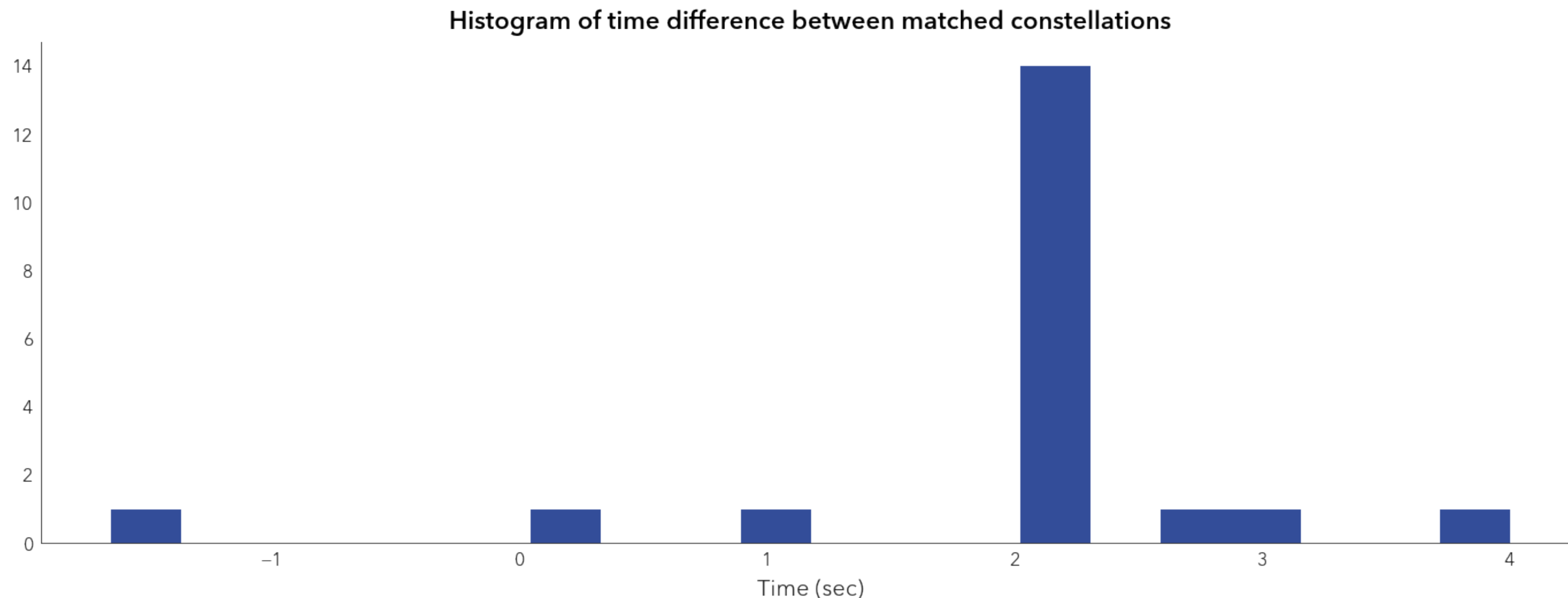
Matching a song snippet

- The query is usually offset and smaller



Looking at matches that matter

- Not all matches will be correct
 - But these that have the same time offset between the files will suggest that the two files are probably the same



Fast and robust performance

- Robust to noise-contaminated sources
 - 90% for 3dB SNR, 80% for 0dB SNR
- Rapid searching due to hashing speedup
 - 20,000 song database: 5-500ms per query

Recap

- Simple sound classifiers
 - Gaussian, Gaussian mixtures
- Generic features for MIR
 - Genre classification
- Audio fingerprinting

Reading material

- Genre classification

- <http://webhome.cs.uvic.ca/~gtzan/work/pubs/tsap02gtzan.pdf>

- Audio fingerprinting

- <https://www.ee.columbia.edu/~dpwe/papers/Wang03-shazam.pdf>

- Audio features

- https://web.archive.org/web/20241010044017/https://recherche.ircam.fr/anasyn/peeters/ARTICLES/Peeters_2003_cuidadoaudiofeatures.pdf

Next lab

- Sound classifiers
 - Using generic features to classify sound files
- Music style classifiers
 - Making classifiers to identify music styles